



Python Developer Quest прокачка до Middle

Модуль 1 - Мир Контейнеров — Лес Структур Данных

Урок 1 - Магия Счётчиков (Counter)

Шаг 01:

Миссия 1

В лесу структур данных вы встретили волшебного гоблина, который считает ягоды. Какой инструмент Python поможет вам быстро посчитать количество каждого вида ягод в корзине?

Подсказка: Подумайте о контейнерах, которые умеют считать элементы автоматически.

Ответ:

- **`collections.Counter`**
- `list`
- `dict`
- `set`

Шаг 02:

Миссия 2

Вы нашли сундук с разноцветными кристаллами. Как с помощью Counter узнать, сколько каждого цвета в сундуке?

Подсказка: Сначала создайте объект Counter с вашей коллекцией, потом смотрите на результат.

Ответ:

- **Counter(кристаллы)**
- len(кристаллы)
- set(кристаллы)
- sum(кристаллы)

🧙 **Шаг 03:**

Миссия 3

Ваш магический питомец принес магические камни. Как с помощью Counter можно получить самый частый камень?

Подсказка: В Counter есть метод, который возвращает элементы по частоте их появления.

Ответ:

- **most_common(1)**
- sum(камни)
- max(камни)
- len(камни)

🧙 **Шаг 04:**

Миссия 4

Формулировка: Сопоставьте метод Counter с его действием.

Методы:

Ответ:

- elements()
- most_common()
- subtract()
- update()

Действия:

1. Возвращает элементы с повторением по их количеству
2. Возвращает элементы в порядке убывания частоты
3. Вычитает элементы другого Counter
4. Добавляет элементы из итерируемого объекта

Подсказка: Вспомните, что некоторые методы изменяют объект, а некоторые возвращают новые значения.

- **elements()** — Возвращает элементы с повторением по их количеству
- **most_common()** — Возвращает элементы в порядке убывания частоты
- **subtract()** — Вычитает элементы другого Counter
- **update()** — Добавляет элементы из итерируемого объекта

🧙 **Шаг 05:**

Миссия 5

Формулировка: Сопоставьте задачу с правильным использованием Counter.

Задачи:

Ответ:

- Посчитать, сколько раз каждый символ встречается в заклинании

- Обновить количество ингредиентов после приготовления зелья
- Узнать три самых редких кристалла в коллекции
- Удалить использованные ягоды из корзины

Методы Counter:

1. `elements()`
2. `most_common(3)`
3. `update()`
4. `subtract()`

Подсказка: Обратите внимание, какие методы возвращают новые значения, а какие изменяют существующий Counter.

- **Посчитать, сколько раз каждый символ встречается в заклинании** — `elements()`
- **Обновить количество ингредиентов после приготовления зелья** — `update()`
- **Узнать три самых редких кристалла в коллекции** — `most_common(3)`
- **Удалить использованные ягоды из корзины** — `subtract()`

🧙 Шаг 06:

Миссия 6

Формулировка: Сопоставьте тип данных, которые можно использовать с Counter, с его действием.

Типы:

Ответ:

- `list`
- `tuple`
- `dict`
- `string`

Использование:

1. Можно считать элементы последовательности по частоте
2. Можно использовать ключи и значения для обновления счетчика
3. Можно посчитать, сколько раз встречается каждая буква
4. Поддерживается итерация по элементам как в списке

Подсказка: Подумайте, какие объекты являются итерируемыми и что Counter умеет принимать на вход.

- **list** — Можно считать элементы последовательности по частоте
- **tuple** — Поддерживается итерация по элементам как в списке
- **dict** — Можно использовать ключи и значения для обновления счетчика
- **string** — Можно посчитать, сколько раз встречается каждая буква

🧙 Шаг 07:

Миссия 7

Вопрос: Расположите шаги использования Counter для анализа слов в заклинании в правильном порядке:

1. Создать Counter с текстом заклинания
2. Вызвать метод `most_common` для анализа частоты слов
3. Обработать результат для отображения наиболее частых слов

4. Собрать текст заклинания в список слов

Подсказка:

Подумайте, что сначала нужно подготовить данные, потом их подсчитать и только потом проанализировать.

Ответ:

- Собрать текст заклинания в список слов
- Создать Counter с текстом заклинания
- Вызвать метод `most_common` для анализа частоты слов
- Обработать результат для отображения наиболее частых слов

Шаг 08:

Миссия 8

Вопрос: Расположите шаги удаления редких ингредиентов из магического зелья:

1. Определить редкие ингредиенты с помощью Counter
2. Создать объект Counter с ингредиентами
3. Вычесть редкие ингредиенты с помощью `subtract()`
4. Проверить результат

Подсказка:

Помните, что сначала нужно собрать данные, потом определить редкие элементы, а потом их удалить.

Ответ:

- Создать объект Counter с ингредиентами
- Определить редкие ингредиенты с помощью Counter
- Вычесть редкие ингредиенты с помощью `subtract()`
- Проверить результат

Шаг 09:

Миссия 9

Вопрос: Расположите шаги создания отчета о частоте встречаемости магических слов:

1. Собрать все слова в список
2. Создать Counter с этим списком
3. Отсортировать результаты по убыванию частоты
4. Вывести отчет в удобной форме

Подсказка:

Сначала собираем данные, затем считаем, сортируем и выводим.

Ответ:

- Собрать все слова в список
- Создать Counter с этим списком
- Отсортировать результаты по убыванию частоты
- Вывести отчет в удобной форме

Шаг 10:

Миссия 10

Вопрос: Расположите шаги комбинирования нескольких коллекций ингредиентов с помощью Counter:

1. Создать Counter для каждой коллекции
2. Объединить Counters с помощью оператора +
3. Проанализировать итоговый Counter
4. Использовать most_common для выявления самых частых ингредиентов

Подсказка:

Сначала создаем отдельные объекты, потом объединяем и анализируем.

Ответ:

- Создать Counter для каждой коллекции
- Объединить Counters с помощью оператора +
- Проанализировать итоговый Counter
- Использовать most_common для выявления самых частых ингредиентов

Урок 2 - Лабиринт defaultdict

Шаг 01:

Миссия 1

В лесу данных вы встретили магического эльфа, который хочет создать словарь с автоматическими значениями. Какой инструмент Python поможет это сделать?

Подсказка: Словарь должен автоматически создавать новые ключи при обращении к ним.

Ответ:

- **collections.defaultdict**
- dict
- Counter
- set

Шаг 02:

Миссия 2

Эльф принес пустые корзины для магических камней. Как создать defaultdict, который будет по умолчанию списком, чтобы добавлять камни без ошибок?

Подсказка: Используйте встроенный тип для коллекций.

Ответ:

- **defaultdict(list)**
- defaultdict(int)
- dict(list)
- set()

Шаг 03:

Миссия 3

Вы хотите подсчитать очки игроков в игре, но не хотите проверять наличие ключей в словаре. Какой тип defaultdict использовать?

Подсказка: Начальное значение должно быть числом.

Ответ:

- **defaultdict(int)**

- defaultdict(list)
- dict(int)
- set()

🧙 Шаг 04:

Миссия 4

Формулировка: Сопоставьте тип данных defaultdict с его типичным применением.

Типы:

Ответ:

- defaultdict(list)
- defaultdict(int)
- defaultdict(set)

Применение:

1. Подсчет количества объектов
2. Группировка элементов по ключам
3. Хранение уникальных значений для каждого ключа

Подсказка: Подумайте, где удобно добавлять элементы без проверки наличия ключа.

- **defaultdict(list)** — Группировка элементов по ключам
- **defaultdict(int)** — Подсчет количества объектов
- **defaultdict(set)** — Хранение уникальных значений для каждого ключа

🧙 Шаг 05:

Миссия 5

Формулировка: Сопоставьте задачу с правильным созданием defaultdict.

Задачи:

Ответ:

- Собрать список учеников по классам
- Посчитать количество монет у игроков
- Хранить уникальные магические заклинания каждого ученика

Способ создания:

1. defaultdict(list)
2. defaultdict(int)
3. defaultdict(set)

Подсказка: Обратите внимание на то, что нужно хранить: списки, числа или уникальные значения.

- **Собрать список учеников по классам** — defaultdict(list)
- **Посчитать количество монет у игроков** — defaultdict(int)
- **Хранить уникальные магические заклинания каждого ученика** — defaultdict(set)

🧙 Шаг 06:

Миссия 6

Формулировка: Сопоставьте действие с эффектом defaultdict.

Действия:

Ответ:

- Обращение к несуществующему ключу
- Добавление элемента в список по ключу
- Увеличение счетчика по ключу

Эффект:

1. Создается новый ключ с указанным значением по умолчанию
2. Элемент добавляется без ошибки
3. Счетчик увеличивается автоматически

Подсказка: Подумайте, какой эффект обеспечивает автоматическое создание ключей.

- **Обращение к несуществующему ключу** — Создается новый ключ с указанным значением по умолчанию
- **Добавление элемента в список по ключу** — Элемент добавляется без ошибки
- **Увеличение счетчика по ключу** — Счетчик увеличивается автоматически

 Шаг 07:

Миссия 7

Вопрос: Расположите шаги использования defaultdict для группировки предметов по категориям:

1. Создать пустой defaultdict с list
2. Перебрать все предметы
3. Добавить предмет в список по категории
4. Проанализировать результат

 Подсказка:

Сначала создаем структуру, затем наполняем и только потом анализируем.

Ответ:

- Создать пустой defaultdict с list
- Перебрать все предметы
- Добавить предмет в список по категории
- Проанализировать результат

 Шаг 08:

Миссия 8

Вопрос: Расположите шаги подсчета очков игроков с defaultdict:

1. Создать defaultdict(int)
2. Перебрать игроков и начислить очки
3. Увеличить счетчик для каждого игрока
4. Проверить итоговые значения

 Подсказка:

Подумайте, как использовать int по умолчанию для удобного подсчета.

Ответ:

- Создать defaultdict(int)
- Перебрать игроков и начислить очки
- Увеличить счетчик для каждого игрока
- Проверить итоговые значения

Шаг 09:

Миссия 9

Вопрос: Расположите шаги для объединения нескольких defaultdict(list) в один:

1. Создать итоговый defaultdict(list)
2. Перебрать каждый исходный defaultdict
3. Добавить все элементы по ключам в итоговый defaultdict
4. Использовать объединенный defaultdict для анализа данных

Подсказка:

Сначала создаем структуру для объединения, потом копируем элементы и анализируем.

Ответ:

- Создать итоговый defaultdict(list)
- Перебрать каждый исходный defaultdict
- Добавить все элементы по ключам в итоговый defaultdict
- Использовать объединенный defaultdict для анализа данных

Шаг 10:

Миссия 10

Вопрос: Расположите шаги создания отчета по уникальным магическим заклинаниям каждого ученика:

1. Создать defaultdict(set)
2. Перебрать учеников и их заклинания
3. Добавить каждое заклинание в set по ключу ученика
4. Проанализировать итоговый набор уникальных заклинаний

Подсказка:

Сначала создаем структуру для хранения уникальных значений, затем наполняем и анализируем.

Ответ:

- Создать defaultdict(set)
- Перебрать учеников и их заклинания
- Добавить каждое заклинание в set по ключу ученика
- Проанализировать итоговый набор уникальных заклинаний

Урок 3 - Очередь заданий (deque + heapq)

Шаг 01:

Миссия 1

В магическом складе задания поступают одно за другим. Какой контейнер Python идеально подходит для обработки очереди FIFO?

Подсказка: Подумайте о структуре, которая позволяет быстро добавлять в конец и удалять с начала.

Ответ:

- collections.deque
- list
- set

- dict

🔮 Шаг 02:

Миссия 2

В лесу заданий некоторые миссии важнее других. Какой контейнер Python позволяет автоматически поддерживать элементы в порядке приоритетов?

Подсказка: Подумайте о структуре, где всегда можно быстро достать минимальный элемент.

Ответ:

- **heapq**
- deque
- list
- set

🔮 Шаг 03:

Миссия 3

Вы хотите добавить новое задание в конец очереди. Какая операция deque поможет сделать это быстро и безопасно?

Подсказка: Смотрите на методы, которые влияют на правый конец очереди.

Ответ:

- **append()**
- pop()
- appendleft()
- popleft()

🔮 Шаг 04:

Миссия 4

Формулировка: Сопоставьте операции deque с их результатом.

Операции:

Ответ:

- append(x)
- appendleft(x)
- pop()
- popleft()

Результат:

1. Добавляет элемент в конец
2. Добавляет элемент в начало
3. Удаляет элемент с конца
4. Удаляет элемент с начала

Подсказка: Вспомните, что у deque есть два конца: левый и правый.

- **append(x)** — Добавляет элемент в конец
- **appendleft(x)** — Добавляет элемент в начало
- **pop()** — Удаляет элемент с конца
- **popleft()** — Удаляет элемент с начала

🔗 Шаг 05:

Миссия 5

Формулировка: Сопоставьте операции `heapq` с их функционалом.

Операции:

Ответ:

- `heappush(heap, x)`
- `heappop(heap)`
- `heapify(list)`

Функционал:

1. Добавляет элемент в кучу, сохраняя порядок
2. Удаляет минимальный элемент из кучи
3. Преобразует обычный список в кучу

Подсказка: Помните, что `heapq` всегда поддерживает минимальный элемент сверху.

- **`heappush(heap, x)`** — Добавляет элемент в кучу, сохраняя порядок
- **`heappop(heap)`** — Удаляет минимальный элемент из кучи
- **`heapify(list)`** — Преобразует обычный список в кучу

🔗 Шаг 06:

Миссия 6

Вопрос: Расположите шаги добавления задания в очередь `deque` и его обработки:

1. Создать пустую `deque`
2. Добавить новые задания в конец
3. Извлечь задание с начала для обработки
4. Отметить задание как выполненное

🔗 Подсказка:

Сначала создаем структуру, затем наполняем и обрабатываем элементы по порядку.

Ответ:

- Создать пустую `deque`
- Добавить новые задания в конец
- Извлечь задание с начала для обработки
- Отметить задание как выполненное

🔗 Шаг 07:

Миссия 7

Вопрос: Расположите шаги работы с приоритетной очередью `heapq`:

1. Создать список заданий
2. Преобразовать список в кучу с `heapify`
3. Добавлять новые задания с `heappush`
4. Извлекать минимальные элементы с `heappop`

🔗 Подсказка:

Сначала готовим структуру, затем наполняем и извлекаем элементы по приоритету.

Ответ:

- Создать список заданий
- Преобразовать список в кучу с `heapify`
- Добавлять новые задания с `heappush`
- Извлекать минимальные элементы с `heappop`

 Шаг 08:

Миссия 8

Вопрос: Расположите шаги объединения двух очередей `deque`:

1. Создать новую пустую `deque`
2. Добавить все элементы первой очереди
3. Добавить все элементы второй очереди
4. Использовать объединенную `deque` для обработки заданий

 Подсказка:

Сначала создаем структуру, потом последовательно наполняем элементами из обеих очередей.

Ответ:

- Создать новую пустую `deque`
- Добавить все элементы первой очереди
- Добавить все элементы второй очереди
- Использовать объединенную `deque` для обработки заданий

 Шаг 09:

Миссия 9

Вопрос: Расположите шаги выполнения всех заданий из приоритетной очереди `heapq`:

1. Создать список заданий с приоритетами
2. Преобразовать список в кучу с `heapify`
3. Постепенно извлекать задания с `heappop`
4. Отметить каждое задание как выполненное

 Подсказка:

Следуйте логике работы приоритетной очереди: подготовка, наполнение, извлечение, выполнение.

Ответ:

- Создать список заданий с приоритетами
- Преобразовать список в кучу с `heapify`
- Постепенно извлекать задания с `heappop`
- Отметить каждое задание как выполненное

Модуль 2 - Башня Функций — Магия Функционального Программирования

Урок 1 - Заклинания `mapfilter`

 Шаг 01:

Миссия 1

В волшебной лаборатории маг преобразует список чисел в их квадраты. Какой инструмент Python ему поможет?

Подсказка: Подумайте о функции, которая применяет действие ко всем элементам сразу.

Ответ:

- **map**
- filter
- reduce
- zip

Шаг 02:

Миссия 2

В замке алхимика нужно оставить только положительные ингредиенты из длинного списка. Какой инструмент Python выбрать?

Подсказка: Функция должна отсеивать элементы по условию.

Ответ:

- **filter**
- map
- zip
- enumerate

Шаг 03:

Миссия 3

Молодой маг хочет одновременно умножить все числа на 2 и оставить только чётные. Какие заклинания Python нужны?

Подсказка: Две функции можно использовать вместе: одна изменяет элементы, другая отбирает их.

Ответ:

- **map**
- **filter**
- reduce
- zip

Шаг 04:

Миссия 4

Формулировка: Сопоставьте функции map и filter с их действиями.

Функции:

Ответ:

- map
- filter

Действия:

1. Применяет функцию ко всем элементам последовательности
2. Отбирает элементы, для которых функция возвращает True

Подсказка: map изменяет значения, filter выбирает элементы.

- **map** — Применяет функцию ко всем элементам последовательности

- **filter** — Отбирает элементы, для которых функция возвращает True

🧙 Шаг 05:

Миссия 5

Формулировка: Сопоставьте задачи с правильной функцией Python.

Задачи:

Ответ:

- Увеличить все уровни персонажей на 1
- Оставить только положительные силы
- Посчитать общее количество магических камней

Функции:

1. map
2. filter
3. reduce

Подсказка: map для изменения, filter для отбора, reduce для агрегирования.

- **Увеличить все уровни персонажей на 1** — map
- **Оставить только положительные силы** — filter
- **Посчитать общее количество магических камней** — reduce

🧙 Шаг 06:

Миссия 6

Формулировка: Сопоставьте шаги использования map/filter для очистки и преобразования данных:

Шаги:

Ответ:

- Создать список чисел
- Применить filter для отбора нужных элементов
- Применить map для изменения значений
- Собрать результат

Подсказка: Сначала отбираем элементы, потом изменяем их, потом получаем итоговый список.

- **Создать список чисел** — Инициализация данных
- **Применить filter для отбора нужных элементов** — Отбор элементов
- **Применить map для изменения значений** — Преобразование элементов
- **Собрать результат** — Формирование итогового списка

🧙 Шаг 07:

Миссия 7

Вопрос: Расположите шаги применения map для возведения всех чисел списка в квадрат:

1. Создать список чисел
2. Определить функцию возведения в квадрат
3. Применить map к списку
4. Преобразовать результат в список и вывести

🧙 Подсказка:

Сначала данные, потом функция, потом map и вывод результата.

Ответ:

- Создать список чисел
- Определить функцию возведения в квадрат
- Применить map к списку
- Преобразовать результат в список и вывести

Шаг 08:

Миссия 8

Вопрос: Расположите шаги использования filter для оставления только чётных чисел:

1. Создать список чисел
2. Определить функцию для проверки чётности
3. Применить filter к списку
4. Преобразовать результат в список и вывести

 **Подсказка:**

Сначала данные, потом функция проверки, затем filter и вывод результата.

Ответ:

- Создать список чисел
- Определить функцию для проверки чётности
- Применить filter к списку
- Преобразовать результат в список и вывести

Шаг 09:

Миссия 9

Вопрос: Расположите шаги комбинированного применения map и filter для увеличения только положительных чисел:

1. Создать список чисел
2. Применить filter для отбора положительных чисел
3. Определить функцию увеличения на 1
4. Применить map к отобранным элементам
5. Преобразовать результат в список и вывести

 **Подсказка:**

Сначала отбираем, потом изменяем, потом выводим результат.

Ответ:

- Создать список чисел
- Применить filter для отбора положительных чисел
- Определить функцию увеличения на 1
- Применить map к отобранным элементам
- Преобразовать результат в список и вывести

Шаг 10:

Миссия 10

Вопрос: Расположите шаги полного анализа списка чисел с фильтрацией и преобразованием:

1. Прочитать список чисел
2. Применить filter для отбора нужных элементов
3. Определить функцию преобразования
4. Применить map к отобранным элементам
5. Собрать и вывести итоговый список

💡 **Подсказка:**

Сначала данные, потом фильтр, потом map, потом формируем результат.

Ответ:

- Прочитать список чисел
- Применить filter для отбора нужных элементов
- Определить функцию преобразования
- Применить map к отобранным элементам
- Собрать и вывести итоговый список

📖 Урок 2 - Декоратор Логирования

💡 **Шаг 01:**

Миссия 1

Молодой маг хочет видеть, когда заклинание выполняется. Какой инструмент Python поможет автоматически логировать вызовы функций?

Подсказка: Подумайте о «обёртке» для функции, которая выполняется при её вызове.

Ответ:

- **Декоратор**
- Генератор
- Класс
- Lambda-функция

💡 **Шаг 02:**

Миссия 2

Маг хочет, чтобы каждая функция в башне оставляла запись о запуске с временем. Какой стандартный модуль Python использовать для логирования?

Подсказка: Подумайте о модуле, который умеет вести журнал событий.

Ответ:

- **logging**
- os
- sys
- timeit

💡 **Шаг 03:**

Миссия 3

Маг решил написать декоратор, чтобы отслеживать входные аргументы функции. Какой элемент декоратора позволяет перехватывать аргументы?

Подсказка: Подумайте о функции внутри функции, которая получает *args и **kwargs.

Ответ:

- **wrapper-функция**
- lambda-функция
- основная функция
- класс

Шаг 04:

Миссия 4

Формулировка: Сопоставьте элементы декоратора логирования с их назначением.

Элементы:

Ответ:

- Декоратор
- Wrapper-функция
- logging

Назначение:

1. Обёртывает целевую функцию и перехватывает вызовы
2. Модуль для ведения журнала событий
3. Функция, добавляющая поведение к другой функции

Подсказка: Сначала определяем функцию-декоратор, потом wrapper, потом используем модуль для записи.

- **Декоратор** — Функция, добавляющая поведение к другой функции
- **Wrapper-функция** — Обёртывает целевую функцию и перехватывает вызовы
- **logging** — Модуль для ведения журнала событий

Шаг 05:

Миссия 5

Формулировка: Сопоставьте ситуации с действиями декоратора логирования.

Ситуации:

Ответ:

- Функция вызвана
- Функция вернула результат
- Функция получила аргументы

Действия декоратора:

1. Записать вызов в лог
2. Записать результат выполнения
3. Записать полученные аргументы

Подсказка: Декоратор логирует вызов, аргументы и результат.

- **Функция вызвана** — Записать вызов в лог
- **Функция вернула результат** — Записать результат выполнения
- **Функция получила аргументы** — Записать полученные аргументы

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте типичные шаги создания декоратора логирования с их описанием.

Шаги:

Ответ:

- Определить декоратор
- Создать `wrapper`-функцию
- Вызвать целевую функцию внутри `wrapper`
- Вернуть `wrapper` из декоратора

Подсказка: Сначала создаём декоратор, потом `wrapper`, потом вызываем функцию, потом возвращаем обёртку.

- **Определить декоратор** — Создание функции-декоратора
- **Создать `wrapper`-функцию** — Обёртывание целевой функции
- **Вызвать целевую функцию внутри `wrapper`** — Выполнение основной логики
- **Вернуть `wrapper` из декоратора** — Возврат обёрнутой функции

🔮 Шаг 07:

Миссия 7

Вопрос: Расположите шаги применения декоратора логирования для функции расчёта опыта персонажа:

1. Создать функцию расчёта опыта
2. Определить декоратор логирования
3. Обёрнуть функцию декоратором
4. Вызвать функцию и проверить лог

🔮 Подсказка:

Сначала создаём функцию, потом декоратор, потом обёртываем, потом вызываем.

Ответ:

- Создать функцию расчёта опыта
- Определить декоратор логирования
- Обёрнуть функцию декоратором
- Вызвать функцию и проверить лог

🔮 Шаг 08:

Миссия 8

Вопрос: Расположите шаги логирования функции перемещения персонажа с аргументами `x` и `y`:

1. Определить декоратор
2. Создать `wrapper` с `*args` и `**kwargs`
3. Записать в лог аргументы `x` и `y`
4. Вызвать исходную функцию внутри `wrapper`
5. Вернуть `wrapper`

🔮 Подсказка:

Обратите внимание на порядок: декоратор, `wrapper`, лог аргументов, вызов функции, возврат `wrapper`.

Ответ:

- Определить декоратор
- Создать wrapper с *args и **kwargs
- Записать в лог аргументы x и y
- Вызвать исходную функцию внутри wrapper
- Вернуть wrapper

 Шаг 09:

Миссия 9

Вопрос: Расположите шаги логирования функции боя с возвратом результата урона:

1. Создать функцию атаки
2. Определить декоратор
3. Создать wrapper
4. Логировать вызов и аргументы
5. Вызвать функцию и сохранить результат
6. Логировать результат
7. Вернуть wrapper

 Подсказка:

Помните: логируем вызов и аргументы до выполнения, результат после выполнения.

Ответ:

- Создать функцию атаки
- Определить декоратор
- Создать wrapper
- Логировать вызов и аргументы
- Вызвать функцию и сохранить результат
- Логировать результат
- Вернуть wrapper

 Шаг 10:

Миссия 10

Вопрос: Расположите шаги создания полного декоратора логирования для всех функций магической башни:

1. Импортировать модуль logging
2. Определить функцию-декоратор
3. Создать wrapper для перехвата аргументов и результатов
4. Добавить записи в лог о вызове и результате
5. Вернуть wrapper из декоратора
6. Применить декоратор ко всем функциям

 Подсказка:

Сначала подготовка модуля, потом создание декоратора и wrapper, логирование, возврат, применение ко всем функциям.

Ответ:

- Импортировать модуль logging
- Определить функцию-декоратор
- Создать wrapper для перехвата аргументов и результатов

- Добавить записи в лог о вызове и результате
- Вернуть `wrapper` из декоратора
- Применить декоратор ко всем функциям

Урок 3 - Генераторы времени

Шаг 01:

Миссия 1

Магическая башня хочет создавать бесконечную последовательность временных меток. Какой инструмент Python для этого идеально подходит?

Подсказка: Подумайте о функции, которая возвращает значения по одному и может продолжаться бесконечно.

Ответ:

- **Генератор**
- Список
- Кортеж
- Множество

Шаг 02:

Миссия 2

Маг хочет, чтобы генератор времени выдавал только каждый второй час. Какой ключевой инструмент поможет реализовать фильтрацию значений генератора?

Подсказка: Подумайте о функции, которая принимает генератор и возвращает только те элементы, которые удовлетворяют условию.

Ответ:

- **filter**
- map
- reduce
- zip

Шаг 03:

Миссия 3

Маг хочет преобразовать генератор временных меток в строки формата 'YYYY-MM-DD HH:MM'. Какой инструмент Python поможет сделать это поэтапно?

Подсказка: Подумайте о функции, которая применяет преобразование к каждому элементу генератора.

Ответ:

- **map**
- filter
- enumerate
- sum

Шаг 04:

Миссия 4

Формулировка: Сопоставьте понятия генераторов времени с их назначением.

Понятия:

Ответ:

- Генератор
- map
- filter
- next()

Назначения:

1. Выдает следующий элемент последовательности
2. Создает последовательность значений по одному
3. Применяет функцию к каждому элементу
4. Отбирает элементы по условию

Подсказка: Сначала думаем о том, кто создает последовательность, кто преобразует, кто фильтрует, кто получает элемент.

- **Генератор** — Создает последовательность значений по одному
- **map** — Применяет функцию к каждому элементу
- **filter** — Отбирает элементы по условию
- **next()** — Выдает следующий элемент последовательности

 Шаг 05:**Миссия 5**

Формулировка: Сопоставьте типичные сценарии работы с генератором времени с правильной функцией.

Сценарии:

Ответ:

- Получить первый элемент
- Преобразовать метки времени в строки
- Отфильтровать метки только для рабочего времени

Функции:

1. map
2. filter
3. next()

Подсказка: Сначала думаем, кто извлекает элемент, кто преобразует, кто фильтрует.

- **Получить первый элемент** — next()
- **Преобразовать метки времени в строки** — map
- **Отфильтровать метки только для рабочего времени** — filter

 Шаг 06:**Миссия 6**

Формулировка: Сопоставьте шаги создания генератора времени с их описанием.

Шаги:

Ответ:

- Определить функцию генератора
- Использовать yield для выдачи значений
- Применить map для преобразования

- Применить `filter` для отбора

Подсказка: Думайте о порядке создания генератора: сначала функция, потом `yield`, потом преобразование и фильтрация.

- **Определить функцию генератора** — Создать функцию, которая будет выдавать значения по одному
- **Использовать `yield` для выдачи значений** — Каждый вызов `next()` возвращает новое значение
- **Применить `map` для преобразования** — Преобразовать элементы генератора, например, в строки
- **Применить `filter` для отбора** — Отобразить только нужные элементы по условию

🔗 Шаг 07:

Миссия 7

Вопрос: Расположите шаги работы с генератором времени для создания списка рабочих часов:

1. Определить генератор часов
2. Использовать `filter` для отбора часов с 9 до 18
3. Преобразовать часы в строки с помощью `map`
4. Собрать элементы в список с `list()`

🔗 Подсказка:

Сначала создаём генератор, потом фильтруем, потом преобразуем, потом собираем в список.

Ответ:

- Определить генератор часов
- Использовать `filter` для отбора часов с 9 до 18
- Преобразовать часы в строки с помощью `map`
- Собрать элементы в список с `list()`

🔗 Шаг 08:

Миссия 8

Вопрос: Расположите шаги генерации временных меток с интервалом 30 минут:

1. Определить функцию генератора
2. Использовать `yield` для выдачи следующей метки времени
3. Применить `filter` для отбора меток по условию
4. Применить `map` для преобразования в строки
5. Использовать `next()` для получения нескольких значений

🔗 Подсказка:

Подумайте о правильной последовательности: генератор → `yield` → фильтр → преобразование → извлечение значений.

Ответ:

- Определить функцию генератора
- Использовать `yield` для выдачи следующей метки времени
- Применить `filter` для отбора меток по условию
- Применить `map` для преобразования в строки
- Использовать `next()` для получения нескольких значений

Шаг 09:

Миссия 9

Вопрос: Расположите шаги создания генератора событий для игры с временными метками:

1. Определить генератор событий
2. Использовать `yield` для передачи события
3. Применить `map` для добавления описания события
4. Применить `filter` для отбора только редких событий
5. Собрать 10 событий в список

Подсказка:

Следуйте логике генератора: генератор → `yield` → `map` → `filter` → сбор элементов.

Ответ:

- Определить генератор событий
- Использовать `yield` для передачи события
- Применить `map` для добавления описания события
- Применить `filter` для отбора только редких событий
- Собрать 10 событий в список

Шаг 10:

Миссия 10

Вопрос: Расположите шаги создания генератора для бесконечной последовательности временных меток с фильтрацией и преобразованием:

1. Определить генератор
2. Использовать `yield` для выдачи меток
3. Применить `filter` для отбора только чётных часов
4. Применить `map` для форматирования в строки
5. Использовать цикл или `next()` для получения значений

Подсказка:

Подумайте о логике генератора: генерация → фильтрация → преобразование → получение элементов.

Ответ:

- Определить генератор
- Использовать `yield` для выдачи меток
- Применить `filter` для отбора только чётных часов
- Применить `map` для форматирования в строки
- Использовать цикл или `next()` для получения значений

Модуль 3 - Хроники Асинхронности — Порталы Времени

Урок 1 - Поток Threading

Шаг 01:

Миссия 1

Магическая лаборатория хочет запускать несколько заклинаний одновременно. Какой инструмент Python для этого идеально подходит?

Подсказка: Подумайте о способе, который позволяет выполнять несколько задач параллельно.

Ответ:

- **threading**
- time.sleep
- while True
- input()

Шаг 02:

Миссия 2

Маг хочет запустить функцию заклинания в отдельном потоке. Какая конструкция Python нужна для создания потока?

Подсказка: Подумайте о классе, который принимает функцию и запускает её параллельно.

Ответ:

- **threading.Thread(target=функция)**
- threading.start()
- time.Thread(функция)
- asyncio.run()

Шаг 03:

Миссия 3

Маг хочет, чтобы все потоки заклинаний завершились перед продолжением эксперимента. Какая команда поможет дождаться завершения потока?

Подсказка: Подумайте о методе, который приостанавливает главный поток до завершения других.

Ответ:

- **join()**
- start()
- sleep()
- run()

Шаг 04:

Миссия 4

Формулировка: Сопоставьте понятия о потоках с их назначением.

Понятия:

Ответ:

- threading.Thread
- start()
- join()
- daemon

Назначения:

1. Запуск потока
2. Создание нового потока выполнения
3. Ожидание завершения потока

4. Позволяет потоку завершиться вместе с основным

Подсказка: Думайте, кто создаёт поток, кто запускает, кто ждёт, а кто завершает автоматически.

- **threading.Thread** — Создание нового потока выполнения
- **start()** — Запуск потока
- **join()** — Ожидание завершения потока
- **daemon** — Позволяет потоку завершиться вместе с основным

Шаг 05:

Миссия 5

Формулировка: Сопоставьте сценарий использования потока с правильным методом.

Сценарии:

Ответ:

- Запустить функцию параллельно
- Ожидать завершения вычисления
- Сделать поток фоновым

Методы:

1. start()
2. join()
3. daemon

Подсказка: Сначала запускаем, потом ждём, а для фонового — ставим специальный флаг.

- **Запустить функцию параллельно** — start()
- **Ожидать завершения вычисления** — join()
- **Сделать поток фоновым** — daemon

Шаг 06:

Миссия 6

Формулировка: Сопоставьте преимущества потоков с их описанием.

Преимущества:

Ответ:

- Параллельное выполнение
- Эффективность при I/O задачах
- Лёгкое создание и управление

Описание:

1. Выполнение нескольких задач одновременно
2. Не блокируют основной поток при ожидании ввода/вывода
3. Простая настройка через threading.Thread

Подсказка: Думайте о том, что дают потоки: скорость, не блокируют и простота.

- **Параллельное выполнение** — Выполнение нескольких задач одновременно
- **Эффективность при I/O задачах** — Не блокируют основной поток при ожидании ввода/вывода
- **Лёгкое создание и управление** — Простая настройка через threading.Thread

🔗 Шаг 07:

Миссия 7

Вопрос: Расположите шаги запуска нескольких потоков для одновременной работы функций:

1. Создать объекты потоков с target=функция
2. Запустить каждый поток с start()
3. Дождаться завершения потоков с join()
4. Проверить результаты выполнения

🔗 Подсказка:

Сначала создаём потоки, затем запускаем, потом ждём их завершения и проверяем результаты.

Ответ:

- Создать объекты потоков с target=функция
- Запустить каждый поток с start()
- Дождаться завершения потоков с join()
- Проверить результаты выполнения

🔗 Шаг 08:

Миссия 8

Вопрос: Расположите шаги использования потоков для параллельной обработки данных:

1. Определить функции обработки
2. Создать потоки с этими функциями
3. Запустить потоки
4. Дождаться их завершения
5. Объединить результаты

🔗 Подсказка:

Сначала определяем работу, создаём потоки, запускаем, ждём, потом собираем результаты.

Ответ:

- Определить функции обработки
- Создать потоки с этими функциями
- Запустить потоки
- Дождаться их завершения
- Объединить результаты

🔗 Шаг 09:

Миссия 9

Вопрос: Расположите шаги создания фоновых потоков для магического чата:

1. Создать поток с target=функция_чата
2. Установить daemon=True
3. Запустить поток
4. Главный поток продолжает работу, поток чата работает параллельно

🔗 Подсказка:

Думайте о порядке: создание → фон → запуск → параллельная работа.

Ответ:

- Создать поток с target=функция_чата
- Установить daemon=True
- Запустить поток
- Главный поток продолжает работу, поток чата работает параллельно

 Шаг 10:

Миссия 10

Вопрос: Расположите шаги безопасного использования потоков при работе с общими ресурсами:

1. Создать Lock объект
2. При доступе к ресурсу использовать lock.acquire()
3. Выполнить операцию с ресурсом
4. Освободить lock с помощью lock.release()
5. Продолжить работу потока

 Подсказка:

Подумайте о последовательности безопасного доступа: создать замок → захват → работа → освобождение → продолжение.

Ответ:

- Создать Lock объект
- При доступе к ресурсу использовать lock.acquire()
- Выполнить операцию с ресурсом
- Освободить lock с помощью lock.release()
- Продолжить работу потока

 Урок 2 - AsyncIO

 Шаг 01:

Миссия 1

Ты попал в Хронопортал и встречаешь первый поток событий. Какой модуль Python поможет тебе управлять асинхронными задачами?

Подсказка: Подумай о слове "async" и как его используют для работы с событиями, не блокируя основной поток.

Ответ:

- **asyncio**
- threading
- multiprocessing
- time

 Шаг 02:

Миссия 2

В портале появляется странный объект — корутина. Что нужно использовать, чтобы её запустить?

Подсказка: Подумай, как Python управляет асинхронным выполнением кода.

Ответ:

- **await внутри async функции**

- `time.sleep()`
- `yield` генератор
- `print()`

🔗 Шаг 03:

Миссия 3

Ты видишь множество задач, которые нужно выполнять одновременно. Какой метод `asyncio` позволит запускать несколько корутин параллельно?

Подсказка: Подумай о том, как объединить несколько задач и дождаться их завершения.

Ответ:

- **`asyncio.gather()`**
- `asyncio.sleep()`
- `asyncio.run_until_complete()`
- `asyncio.create_task()`

🔗 Шаг 04:

Миссия 4

Формулировка: Сопоставьте термины `AsyncIO` с их правильным объяснением.

Понятия:

Ответ:

- `async def`
- `await`
- `coroutine`
- `event loop`

Определения:

1. Оператор для ожидания завершения асинхронной задачи
2. Функция, которая может выполняться асинхронно
3. Объект, представляющий асинхронную задачу
4. Центральный цикл, который управляет выполнением корутин

Подсказка: Подумай, что делает каждая конструкция: объявляет функцию, ждёт её, запускает цикл событий или создаёт объект задачи.

- **`async def`** — Функция, которая может выполняться асинхронно
- **`await`** — Оператор для ожидания завершения асинхронной задачи
- **`coroutine`** — Объект, представляющий асинхронную задачу
- **`event loop`** — Центральный цикл, который управляет выполнением корутин

🔗 Шаг 05:

Миссия 5

Формулировка: Сопоставьте методы `asyncio` с их назначением.

Понятия:

Ответ:

- `asyncio.create_task()`
- `asyncio.run()`
- `asyncio.sleep()`

- `asyncio.gather()`

Определения:

1. Запускает главную корутину и завершает event loop после её выполнения
2. Создаёт задачу для параллельного выполнения корутины
3. Объединяет несколько корутин и ждёт их завершения
4. Асинхронная пауза без блокировки event loop

Подсказка: Подумай, что делает каждая функция: создаёт задачу, запускает цикл событий, объединяет корутины или делает паузу.

- **`asyncio.create_task()`** — Создаёт задачу для параллельного выполнения корутины
- **`asyncio.run()`** — Запускает главную корутину и завершает event loop после её выполнения
- **`asyncio.sleep()`** — Асинхронная пауза без блокировки event loop
- **`asyncio.gather()`** — Объединяет несколько корутин и ждёт их завершения

Шаг 06:

Миссия 6

Формулировка: Сопоставьте исключения `asyncio` с их типичной причиной возникновения.

Понятия:

Ответ:

- `asyncio.TimeoutError`
- `RuntimeError`
- `CancelledError`
- `InvalidStateError`

Определения:

1. Задача была отменена до завершения
2. Слишком долго выполнялась операция с ожиданием
3. Попытка выполнить операцию в неправильном состоянии объекта
4. Ошибка при неправильном использовании event loop или корутины

Подсказка: Подумай, какая ошибка возникает при тайм-ауте, отмене задачи или неверном состоянии.

- **`asyncio.TimeoutError`** — Слишком долго выполнялась операция с ожиданием
- **`RuntimeError`** — Ошибка при неправильном использовании event loop или корутины
- **`CancelledError`** — Задача была отменена до завершения
- **`InvalidStateError`** — Попытка выполнить операцию в неправильном состоянии объекта

Шаг 07:

Миссия 7

Вопрос: Расположите шаги выполнения нескольких корутин с `asyncio` в правильном порядке:

1. Создать задачи для всех корутин
2. Запустить event loop с главной корутиной
3. Дождаться завершения всех задач
4. Обработать результаты или ошибки

 Подсказка:

Сначала создаём задачи, потом запускаем loop, ждём завершения, затем обрабатываем результаты.

Ответ:

- Создать задачи для всех корутин
- Запустить event loop с главной корутиной
- Дождаться завершения всех задач
- Обработать результаты или ошибки

 Шаг 08:

Миссия 8

Вопрос: Расположите этапы отладки асинхронного кода в правильном порядке:

1. Добавление логирования внутри корутин
2. Запуск кода и наблюдение за поведением
3. Исправление ошибок на основе логов
4. Повторный запуск и проверка исправлений

 Подсказка:

Логирование помогает понять порядок выполнения и найти ошибки, затем исправляем и проверяем снова.

Ответ:

- Добавление логирования внутри корутин
- Запуск кода и наблюдение за поведением
- Исправление ошибок на основе логов
- Повторный запуск и проверка исправлений

 Шаг 09:

Миссия 9

Вопрос: Расположите последовательность действий при комбинировании asyncio с внешними API:

1. Создать асинхронные запросы к API
2. Запустить их с помощью asyncio.gather()
3. Дождаться всех ответов
4. Обработать полученные данные

 Подсказка:

Сначала формируем запросы, объединяем их, ждём завершения, затем обрабатываем результат.

Ответ:

- Создать асинхронные запросы к API
- Запустить их с помощью asyncio.gather()
- Дождаться всех ответов
- Обработать полученные данные

 Шаг 10:

Миссия 10

Вопрос: Расположите шаги по безопасной отмене асинхронных задач:

1. Отправить сигнал отмены задачи

2. Дождаться обработки `CancelledError` внутри корутины
3. Очистить ресурсы, если нужно
4. Подтвердить завершение задачи

🔔 Подсказка:

Важно сначала отправить сигнал, потом дождаться корректного завершения и освобождения ресурсов.

Ответ:

- Отправить сигнал отмены задачи
- Дождаться обработки `CancelledError` внутри корутины
- Очистить ресурсы, если нужно
- Подтвердить завершение задачи

📖 Урок 3 - Multiprocessing

🔔 Шаг 01:

Миссия 1

Ты попал в Лабораторию Многопроцессности. Какой модуль Python поможет тебе создавать отдельные процессы?

Подсказка: Подумай о слове "process" и о том, как Python может выполнять код параллельно в отдельных потоках ОС.

Ответ:

- **multiprocessing**
- asyncio
- threading
- time

🔔 Шаг 02:

Миссия 2

Ты создаёшь процесс с функцией `worker`. Какой метод нужно вызвать, чтобы процесс начал работать?

Подсказка: Подумай, как в `threading.Thread` запускался поток.

Ответ:

- **start()**
- run()
- execute()
- spawn()

🔔 Шаг 03:

Миссия 3

Процесс завершил работу. Какой метод позволит дождаться его окончания перед продолжением программы?

Подсказка: Подумай о синхронизации, чтобы главный процесс не ушёл дальше раньше времени.

Ответ:

- **join()**
- start()

- `run()`
- `wait()`

🔮 Шаг 04:

Миссия 4

Формулировка: Сопоставьте термины multiprocessing с их объяснением.

Понятия:

Ответ:

- `Process`
- `Queue`
- `Pipe`
- `Pool`

Определения:

1. Класс для создания отдельного процесса
2. Объект для безопасной передачи данных между процессами
3. Механизм связи между двумя процессами
4. Менеджер группы процессов для параллельного выполнения задач

Подсказка: Подумай, что создаёт процесс, что передаёт данные, а что управляет группой процессов.

- **`Process`** — Класс для создания отдельного процесса
- **`Queue`** — Объект для безопасной передачи данных между процессами
- **`Pipe`** — Механизм связи между двумя процессами
- **`Pool`** — Менеджер группы процессов для параллельного выполнения задач

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставьте методы `Process` с их назначением.

Понятия:

Ответ:

- `start()`
- `join()`
- `is_alive()`
- `terminate()`

Определения:

1. Запускает процесс
2. Ожидает завершения процесса
3. Проверяет, выполняется ли процесс
4. Принудительно завершает процесс

Подсказка: Подумай, что запускает процесс, ждёт его, проверяет состояние и завершает.

- **`start()`** — Запускает процесс
- **`join()`** — Ожидает завершения процесса
- **`is_alive()`** — Проверяет, выполняется ли процесс
- **`terminate()`** — Принудительно завершает процесс

🔧 Шаг 06:

Миссия 6

Формулировка: Сопоставьте типы объектов для обмена данными между процессами с их характеристиками.

Понятия:

Ответ:

- Queue
- Pipe
- Manager.dict()
- Value

Определения:

1. Очередь для безопасной передачи объектов между процессами
2. Двухнаправленный канал между двумя процессами
3. Словарь, разделяемый между процессами
4. Объект для хранения одного значения, доступного из разных процессов

Подсказка: Подумай, что передаёт коллекцию, что значение, а что соединяет два процесса.

- **Queue** — Очередь для безопасной передачи объектов между процессами
- **Pipe** — Двухнаправленный канал между двумя процессами
- **Manager.dict()** — Словарь, разделяемый между процессами
- **Value** — Объект для хранения одного значения, доступного из разных процессов

🔧 Шаг 07:

Миссия 7

Вопрос: Расположите шаги создания и запуска нескольких процессов с Pool в правильном порядке:

1. Создать объект Pool с числом процессов
2. Отправить задачи в пул с помощью map/apply
3. Закрыть пул для новых задач
4. Дождаться завершения всех процессов

🔧 Подсказка:

Сначала создаём пул, потом запускаем задачи, закрываем пул и ждём завершения.

Ответ:

- Создать объект Pool с числом процессов
- Отправить задачи в пул с помощью map/apply
- Закрыть пул для новых задач
- Дождаться завершения всех процессов

🔧 Шаг 08:

Миссия 8

Вопрос: Расположите шаги безопасного завершения процессов вручную:

1. Отправить сигнал terminate()
2. Проверить, что процесс завершился с is_alive()
3. Очистить разделяемые ресурсы (Queue, Pipe)

4. Подтвердить окончание работы всех процессов

 Подсказка:

Сначала сигнал, потом проверка состояния и очистка ресурсов.

Ответ:

- Отправить сигнал `terminate()`
- Проверить, что процесс завершился с `is_alive()`
- Очистить разделяемые ресурсы (`Queue`, `Pipe`)
- Подтвердить окончание работы всех процессов

 Шаг 09:

Миссия 9

Вопрос: Расположите этапы обмена данными между процессами с использованием `Queue`:

1. Создать `Queue`
2. Процесс помещает данные в очередь с `put()`
3. Другой процесс получает данные с `get()`
4. Обработать полученные данные

 Подсказка:

Сначала создаём очередь, потом пишем и читаем данные, после чего обрабатываем.

Ответ:

- Создать `Queue`
- Процесс помещает данные в очередь с `put()`
- Другой процесс получает данные с `get()`
- Обработать полученные данные

 Шаг 10:

Миссия 10

Вопрос: Расположите шаги при работе с `Manager.dict()` для нескольких процессов:

1. Создать `Manager` и `dict()`
2. Процессы записывают данные в `dict`
3. Дождаться завершения всех процессов
4. Прочитать и использовать собранные данные

 **Подсказка:** Сначала создаём общий словарь, затем пишем в него из процессов, ждём завершения и используем данные.

Ответ:

- Создать `Manager` и `dict()`
- Процессы записывают данные в `dict`
- Дождаться завершения всех процессов
- Прочитать и использовать собранные данные

Модуль 4 - Подземелья Исключений

Урок 1 - Магия tryexcept

Шаг 01:

Миссия 1

Ты находишься в Лабиринте Ошибок. Какой блок кода позволяет Python обработать исключение и продолжить работу программы?

Подсказка: Подумай о словах 'try' и 'except' — они магическим образом ловят ошибки.

Ответ:

- **try/except**
- if/else
- while/for
- def/function

Шаг 02:

Миссия 2

В бою с Лордом Исключением ты хочешь поймать ошибку деления на ноль. Какой тип исключения нужно указать после except?

Подсказка: Подумай, какая ошибка возникает, когда делишь число на ноль.

Ответ:

- **ZeroDivisionError**
- ValueError
- TypeError
- IndexError

Шаг 03:

Миссия 3

Если ты хочешь поймать любые исключения в бою, какой способ использовать?

Подсказка: Иногда магия без конкретики тоже помогает, но будь осторожен.

Ответ:

- **except Exception**
- except ZeroDivisionError
- except pass
- except None

Шаг 04:

Миссия 4

Формулировка: Сопоставьте термины try/except с их объяснением.

Понятия:

Ответ:

- try
- except
- finally

- else

Определения:

1. Блок кода, который выполняется только если ошибок не было
2. Попытка выполнить код, где может возникнуть ошибка
3. Блок кода, который выполняется при возникновении исключения
4. Блок кода, который выполняется в любом случае

Подсказка: Подумай, какой блок ловит ошибку, какой проверяет успешное выполнение и какой выполняется всегда.

- **try** — Попытка выполнить код, где может возникнуть ошибка
- **except** — Блок кода, который выполняется при возникновении исключения
- **else** — Блок кода, который выполняется только если ошибок не было
- **finally** — Блок кода, который выполняется в любом случае

Шаг 05:

Миссия 5

Формулировка: Сопоставьте тип исключения с примером ситуации.

Понятия:

Ответ:

- IndexError
- KeyError
- ValueError
- ZeroDivisionError

Определения:

1. Деление на ноль
2. Попытка получить элемент списка по несуществующему индексу
3. Неверное значение, например `int('abc')`
4. Обращение к несуществующему ключу в словаре

Подсказка: Подумай, что происходит при делении, при работе с индексом, ключом и преобразованием типов.

- **IndexError** — Попытка получить элемент списка по несуществующему индексу
- **KeyError** — Обращение к несуществующему ключу в словаре
- **ValueError** — Неверное значение, например `int('abc')`
- **ZeroDivisionError** — Деление на ноль

Шаг 06:

Миссия 6

Формулировка: Сопоставьте действия с их результатом при обработке исключений.

Понятия:

Ответ:

- `try: x = 1/0 except ZeroDivisionError: pass`
- `try: int('abc') except ValueError: x = 0`
- `try: d['key'] except KeyError: d['key'] = 1`
- `try: print('ok') except Exception: pass else: print('success')`

Определения:

1. Ошибка перехвачена и пропущена
2. Преобразование неудачно, значение исправлено
3. Ключ добавлен, ошибки нет
4. Код выполнен успешно, вывод 'success'

Подсказка: Подумай, что происходит при ошибке и без ошибки.

- **try: x = 1/0 except ZeroDivisionError: pass** — Ошибка перехвачена и пропущена
- **try: int('abc') except ValueError: x = 0** — Преобразование неудачно, значение исправлено
- **try: d['key'] except KeyError: d['key'] = 1** — Ключ добавлен, ошибки нет
- **try: print('ok') except Exception: pass else: print('success')** — Код выполнен успешно, вывод 'success'

Шаг 07:

Миссия 7

Вопрос: Расположите шаги обработки исключения в правильном порядке:

1. Попытка выполнить код
2. Возникновение ошибки
3. Переход в блок except
4. Выполнение кода после блока try/except

Подсказка:

Сначала программа пытается выполнить код, потом может возникнуть ошибка, затем срабатывает except, и в конце продолжается выполнение.

Ответ:

- Попытка выполнить код
- Возникновение ошибки
- Переход в блок except
- Выполнение кода после блока try/except

Шаг 08:

Миссия 8

Вопрос: Расположите последовательность блоков try/except/else/finally:

1. try: основной код
2. except: обработка ошибок
3. else: выполняется если ошибок нет
4. finally: выполняется всегда

Подсказка:

Подумай, какой блок обрабатывает ошибки, какой выполняется при успехе, а какой — в любом случае.

Ответ:

- try: основной код
- except: обработка ошибок
- else: выполняется если ошибок нет
- finally: выполняется всегда

Шаг 09:

Миссия 9

Вопрос: Расположите шаги обработки нескольких исключений:

1. Попытка выполнить код
2. except TypeError
3. except ValueError
4. finally блок

Подсказка:

Сначала код выполняется, потом проверяем по типу ошибки, и в конце выполняется finally.

Ответ:

- Попытка выполнить код
- except TypeError
- except ValueError
- finally блок

Шаг 10:

Миссия 10

Вопрос: Расположите шаги безопасного использования внешних ресурсов с try/finally:

1. Открытие файла или соединения
2. Попытка выполнить операции с ресурсом
3. Обработка ошибок (при необходимости)
4. Закрытие ресурса в finally

Подсказка:

Сначала открываем ресурс, работаем с ним, ловим ошибки и обязательно закрываем.

Ответ:

- Открытие файла или соединения
- Попытка выполнить операции с ресурсом
- Обработка ошибок (при необходимости)
- Закрытие ресурса в finally

Урок 2 - Логирование событий

Шаг 01:

Миссия 1

Ты попал в Зал Логов. Какой модуль Python помогает записывать события и ошибки программы?

Подсказка: Этот модуль позволяет магически отслеживать, что происходит внутри кода.

Ответ:

- logging
- os
- sys
- time

🔧 Шаг 02:

Миссия 2

Ты хочешь, чтобы все предупреждения и ошибки сохранялись в файл 'game.log'. Какая команда логирования подойдет?

Подсказка: Подумай о функции, которая настраивает логирование сразу для файла.

Ответ:

- **logging.basicConfig(filename='game.log', level=logging.WARNING)**
- logging.file('game.log')
- log.open('game.log')
- logging.save('game.log')

🔧 Шаг 03:

Миссия 3

Ты хочешь, чтобы логировалась вся информация, от отладочной до критической. Какой уровень логирования выбрать?

Подсказка: Уровни идут от DEBUG до CRITICAL.

Ответ:

- **DEBUG**
- INFO
- WARNING
- ERROR

🔧 Шаг 04:

Миссия 4

Формулировка: Сопоставьте метод логирования с его назначением.

Понятия:

Ответ:

- logging.debug()
- logging.info()
- logging.warning()
- logging.error()
- logging.critical()

Определения:

1. Информация для отладки кода
2. Обычные информационные сообщения
3. Предупреждения о возможных проблемах
4. Ошибки, которые мешают работе программы
5. Критические ошибки, приводящие к аварийной остановке

Подсказка: Подумай, какой уровень логирования идет от низкого к высокому по важности.

- **logging.debug()** — Информация для отладки кода
- **logging.info()** — Обычные информационные сообщения
- **logging.warning()** — Предупреждения о возможных проблемах
- **logging.error()** — Ошибки, которые мешают работе программы
- **logging.critical()** — Критические ошибки, приводящие к аварийной остановке

🔗 Шаг 05:

Миссия 5

Формулировка: Сопоставьте параметры basicConfig с их действием.

Понятия:

Ответ:

- filename
- filemode
- level
- format

Определения:

1. Имя файла для сохранения логов
2. Режим открытия файла ('w' или 'a')
3. Минимальный уровень логирования
4. Структура записи лога (шаблон сообщения)

Подсказка: Подумай, что отвечает за имя, режим, уровень и формат сообщения.

- **filename** — Имя файла для сохранения логов
- **filemode** — Режим открытия файла ('w' или 'a')
- **level** — Минимальный уровень логирования
- **format** — Структура записи лога (шаблон сообщения)

🔗 Шаг 06:

Миссия 6

Формулировка: Сопоставьте формат строки лога с его содержимым.

Понятия:

Ответ:

- '%(asctime)s - %(levelname)s - %(message)s'
- '%(levelname)s: %(message)s'
- '%(message)s'

Определения:

1. Полная информация: время, уровень и сообщение
2. Краткая информация: уровень и сообщение
3. Только текст сообщения

Подсказка: Формат определяет, что будет показано в логе.

- **'%(asctime)s - %(levelname)s - %(message)s'** — Полная информация: время, уровень и сообщение
- **'%(levelname)s: %(message)s'** — Краткая информация: уровень и сообщение
- **'%(message)s'** — Только текст сообщения

🔗 Шаг 07:

Миссия 7

Вопрос: Расположите шаги настройки логирования в правильном порядке:

1. Импорт модуля logging

2. Настройка basicConfig с filename, level и format
3. Использование logging.debug/info/warning/error/critical для записи событий
4. Чтение или анализ логов при необходимости

 Подсказка:

Сначала импортируем модуль, затем настраиваем его, используем для записи и только потом анализируем.

Ответ:

- Импорт модуля logging
- Настройка basicConfig с filename, level и format
- Использование logging.debug/info/warning/error/critical для записи событий
- Чтение или анализ логов при необходимости

 Шаг 08:

Миссия 8

Вопрос: Расположите уровни логирования по возрастанию важности:

1. DEBUG
2. INFO
3. WARNING
4. ERROR
5. CRITICAL

 Подсказка:

Начни с отладки и закончи критической аварией.

Ответ:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

 Шаг 09:

Миссия 9

Вопрос: Расположите шаги логирования с использованием try/except:

1. try: попытка выполнить код
2. except: запись ошибки в лог
3. finally: выполнение завершающих действий
4. Использование лога для анализа после выполнения

 Подсказка:

Сначала пробуем, потом ловим ошибки, затем завершаем и анализируем.

Ответ:

- try: попытка выполнить код
- except: запись ошибки в лог
- finally: выполнение завершающих действий
- Использование лога для анализа после выполнения

Шаг 10:

Миссия 10

Вопрос: Расположите шаги настройки ротации логов с TimedRotatingFileHandler:

1. Импорт Handler и logging
2. Создание экземпляра TimedRotatingFileHandler с interval и backupCount
3. Настройка formatter и добавление его к handler
4. Добавление handler к logger

Подсказка:

Сначала импорт, потом создаем обработчик, настраиваем формат и подключаем к логгеру.

Ответ:

- Импорт Handler и logging
- Создание экземпляра TimedRotatingFileHandler с interval и backupCount
- Настройка formatter и добавление его к handler
- Добавление handler к logger

Урок 3 - Тестирование

Шаг 01:

Миссия 1

Ты отправляешься в Лабораторию Тестов. Какой модуль Python используется для написания юнит-тестов?

Подсказка: Этот модуль позволяет проверять функции на правильность и ловить ошибки до запуска программы.

Ответ:

- **unittest**
- sys
- random
- logging

Шаг 02:

Миссия 2

Ты хочешь проверить, что функция возвращает правильное значение. Какой метод unittest позволяет проверить равенство ожидаемого и реального результата?

Подсказка: Название метода намекает на проверку равенства двух объектов.

Ответ:

- **assertEqual**
- assertTrue
- assertFalse
- assertRaises

Шаг 03:

Миссия 3

Твой тест должен проверить, что функция вызывает исключение ValueError. Какой метод использовать?

Подсказка: Метод проверяет, что код внутри блока выбросит определённое исключение.

Ответ:

- **assertRaises**
- **assertEqual**
- **assertTrue**
- **assertFalse**

Шаг 04:

Миссия 4

Формулировка: Сопоставьте методы unittest с их назначением.

Понятия:

Ответ:

- **assertEqual(a, b)**
- **assertTrue(x)**
- **assertFalse(x)**
- **assertRaises(Exception, func)**

Определения:

1. Проверяет равенство двух значений
2. Проверяет, что выражение истинно
3. Проверяет, что выражение ложно
4. Проверяет, что вызываемая функция вызывает исключение

Подсказка: Подумай о названии методов: оно обычно говорит о том, что проверяется.

- **assertEqual(a, b)** — Проверяет равенство двух значений
- **assertTrue(x)** — Проверяет, что выражение истинно
- **assertFalse(x)** — Проверяет, что выражение ложно
- **assertRaises(Exception, func)** — Проверяет, что вызываемая функция вызывает исключение

Шаг 05:

Миссия 5

Формулировка: Сопоставьте типы тестов с их описанием.

Понятия:

Ответ:

- Unit-тест
- Integration-тест
- Functional-тест
- Regression-тест

Определения:

1. Проверяет отдельную функцию или метод
2. Проверяет взаимодействие нескольких компонентов
3. Проверяет работу приложения с точки зрения пользователя
4. Проверяет, что старые ошибки не вернулись после изменений

Подсказка: Подумай о масштабе теста: от маленького блока к полной функциональности.

- **Unit-тест** — Проверяет отдельную функцию или метод

- **Integration-тест** — Проверяет взаимодействие нескольких компонентов
- **Functional-тест** — Проверяет работу приложения с точки зрения пользователя
- **Regression-тест** — Проверяет, что старые ошибки не вернулись после изменений

Шаг 06:

Миссия 6

Формулировка: Сопоставьте элементы unittest с их назначением.

Понятия:

Ответ:

- setUp()
- tearDown()
- TestCase
- TestSuite

Определения:

1. Метод, который выполняется перед каждым тестом
2. Метод, который выполняется после каждого теста
3. Класс, содержащий набор тестов
4. Коллекция нескольких TestCase для запуска вместе

Подсказка: setUp — подготовка, tearDown — завершение, TestCase — отдельный набор тестов, TestSuite — группа.

- **setUp()** — Метод, который выполняется перед каждым тестом
- **tearDown()** — Метод, который выполняется после каждого теста
- **TestCase** — Класс, содержащий набор тестов
- **TestSuite** — Коллекция нескольких TestCase для запуска вместе

Шаг 07:

Миссия 7

Вопрос: Расположите шаги написания теста в правильном порядке:

1. Импорт unittest
2. Создание класса TestCase
3. Определение методов setUp() и tearDown()
4. Написание тестов с assert
5. Запуск тестов через unittest.main()

 **Подсказка:**

Сначала подготовка модуля и класса, затем написание тестов и запуск.

Ответ:

- Импорт unittest
- Создание класса TestCase
- Определение методов setUp() и tearDown()
- Написание тестов с assert
- Запуск тестов через unittest.main()

Шаг 08:

Миссия 8

Вопрос: Расположите типы тестов по уровню охвата приложения:

1. Unit-тест
2. Integration-тест
3. Functional-тест
4. Regression-тест

💡 **Подсказка:**

Сначала маленький блок, затем взаимодействие компонентов, функциональность, и контроль старых ошибок.

Ответ:

- Unit-тест
- Integration-тест
- Functional-тест
- Regression-тест

💡 **Шаг 09:**

Миссия 9

Вопрос: Расположите шаги тестирования с использованием try/except:

1. Попытка выполнить код в try
2. Ловля ошибок в except и запись в лог
3. Выполнение завершающих действий в finally
4. Проверка результатов и анализ ошибок

💡 **Подсказка:**

Сначала пробуем, затем ловим ошибки, завершаем блок и анализируем.

Ответ:

- Попытка выполнить код в try
- Ловля ошибок в except и запись в лог
- Выполнение завершающих действий в finally
- Проверка результатов и анализ ошибок

💡 **Шаг 10:**

Миссия 10

Вопрос: Расположите шаги автоматизации тестов для CI/CD:

1. Написание тестов с unittest или pytest
2. Добавление тестов в репозиторий
3. Настройка CI/CD для автоматического запуска
4. Анализ результатов и исправление ошибок

💡 **Подсказка:**

Сначала пишем тесты, затем добавляем их в систему, настраиваем автоматизацию и смотрим результаты.

Ответ:

- Написание тестов с unittest или pytest
- Добавление тестов в репозиторий
- Настройка CI/CD для автоматического запуска

- Анализ результатов и исправление ошибок

Модуль 5 - Хранилище Артефактов

Урок 1 - JSON и CSV

Шаг 01:

Миссия 1

Ты нашёл таинственный файл с артефактами. Какой модуль Python позволяет работать с форматом JSON?

Подсказка: Этот модуль умеет сериализовать и десериализовать данные в виде текста.

Ответ:

- **json**
- csv
- os
- sys

Шаг 02:

Миссия 2

Ты хочешь сохранить список магических предметов в JSON-файл. Какая функция используется для записи Python-объекта в JSON?

Подсказка: Слово в функции начинается с 'dump'.

Ответ:

- **json.dump**
- json.load
- csv.writer
- open

Шаг 03:

Миссия 3

Ты хочешь прочитать магическую книгу артефактов в формате CSV. Какой модуль поможет это сделать?

Подсказка: Он работает с табличными данными и умеет читать и писать строки через разделители.

Ответ:

- **csv**
- json
- pandas
- os

Шаг 04:

Миссия 4

Формулировка: Сопоставьте функцию с её назначением.

Понятия:

Ответ:

- `json.dump`
- `json.load`
- `csv.writer`
- `csv.reader`

Определения:

1. Записывает Python-объект в JSON-файл
2. Читает Python-объект из JSON-файла
3. Записывает строки в CSV-файл
4. Читает строки из CSV-файла

Подсказка: `dump` и `load` — для JSON, `writer` и `reader` — для CSV.

- **`json.dump`** — Записывает Python-объект в JSON-файл
- **`json.load`** — Читает Python-объект из JSON-файла
- **`csv.writer`** — Записывает строки в CSV-файл
- **`csv.reader`** — Читает строки из CSV-файла

 Шаг 05:

Миссия 5

Формулировка: Сопоставьте аргументы функций с их ролью.

Понятия:

Ответ:

- `json.dump(obj, file)`
- `json.load(file)`
- `csv.writer(file, delimiter)`
- `csv.reader(file, delimiter)`

Определения:

1. Объект Python, который нужно сохранить
2. Файл для чтения или записи
3. Символ-разделитель для CSV

Подсказка: Смотри на порядок аргументов и на их назначение: что сохраняем, что читаем, какой разделитель используем.

- **`obj`** — Объект Python, который нужно сохранить
- **`file`** — Файл для чтения или записи
- **`delimiter`** — Символ-разделитель для CSV

 Шаг 06:

Миссия 6

Формулировка: Сопоставьте типы файлов с их характеристиками.

Понятия:

Ответ:

- JSON
- CSV

Определения:

1. Структурированный текст с ключами и значениями, поддерживает вложенные объекты
2. Табличный текст с разделителями, удобен для списков и таблиц

Подсказка: JSON — для структурированных объектов, CSV — для таблиц.

- **JSON** — Структурированный текст с ключами и значениями, поддерживает вложенные объекты
- **CSV** — Табличный текст с разделителями, удобен для списков и таблиц

Шаг 07:

Миссия 7

Вопрос: Расположите шаги сохранения данных в JSON-файл:

1. Открыть файл с помощью `open()`
2. Вызвать `json.dump()` для записи объекта
3. Закрыть файл

Подсказка:

Сначала подготовка файла, затем запись, потом закрытие.

Ответ:

- Открыть файл с помощью `open()`
- Вызвать `json.dump()` для записи объекта
- Закрыть файл

Шаг 08:

Миссия 8

Вопрос: Расположите шаги чтения данных из CSV-файла:

1. Открыть файл с помощью `open()`
2. Создать объект `csv.reader`
3. Пройтись по строкам через цикл
4. Закрыть файл

Подсказка:

Сначала подготовка, потом чтение строк, потом закрытие.

Ответ:

- Открыть файл с помощью `open()`
- Создать объект `csv.reader`
- Пройтись по строкам через цикл
- Закрыть файл

Шаг 09:

Миссия 9

Вопрос: Расположите шаги конвертации CSV в JSON:

1. Прочитать CSV через `csv.reader`
2. Сформировать список словарей Python
3. Открыть JSON-файл для записи
4. Записать данные через `json.dump()`

5. Закрыть JSON-файл

Подсказка:

Сначала читаем CSV, формируем структуру, открываем JSON и сохраняем.

Ответ:

- Прочитать CSV через `csv.reader`
- Сформировать список словарей Python
- Открыть JSON-файл для записи
- Записать данные через `json.dump()`
- Закрыть JSON-файл

Шаг 10:

Миссия 10

Вопрос: Расположите шаги обработки JSON и CSV для анализа артефактов:

1. Прочитать JSON-файл через `json.load()`
2. Если нужно, конвертировать в CSV через `csv.writer`
3. Обработать данные в Python
4. Сохранить изменения при необходимости

Подсказка:

Сначала читаем данные, затем при необходимости конвертируем, обрабатываем и сохраняем.

Ответ:

- Прочитать JSON-файл через `json.load()`
- Если нужно, конвертировать в CSV через `csv.writer`
- Обработать данные в Python
- Сохранить изменения при необходимости

Урок 2 - SQLite

Шаг 01:

Миссия 1

Ты нашёл древнюю базу артефактов. Какой модуль Python используется для работы с SQLite?

Подсказка: Этот модуль встроен в Python и не требует установки.

Ответ:

- **sqlite3**
- `pymysql`
- `psycopg2`
- `os`

Шаг 02:

Миссия 2

Ты хочешь создать базу артефактов. Какая команда SQLite создаёт соединение с базой данных?

Подсказка: Начинается с `'connect'` и возвращает объект соединения.

Ответ:

- **sqlite3.connect()**
- `sqlite3.execute()`

- `sqlite3.commit()`
- `sqlite3.cursor()`

🔮 Шаг 03:

Миссия 3

Ты хочешь создать таблицу артефактов с колонками 'name' и 'power'. Какая команда используется для выполнения SQL-запроса в Python?

Подсказка: Сначала получаем курсор, затем вызываем метод для выполнения SQL.

Ответ:

- **`cursor.execute()`**
- `sqlite3.commit()`
- `sqlite3.fetchall()`
- `cursor.connect()`

🔮 Шаг 04:

Миссия 4

Формулировка: Сопоставьте объект SQLite с его ролью.

Понятия:

Ответ:

- Connection
- Cursor

Определения:

1. Представляет соединение с базой данных
2. Позволяет выполнять SQL-запросы и получать результаты

Подсказка: Connection — путь к базе, Cursor — инструмент работы с данными.

- **Connection** — Представляет соединение с базой данных
- **Cursor** — Позволяет выполнять SQL-запросы и получать результаты

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставьте методы объекта Connection с их назначением.

Понятия:

Ответ:

- `commit()`
- `close()`

Определения:

1. Сохраняет изменения в базе данных
2. Закрывает соединение с базой

Подсказка: Commit — фиксация изменений, close — завершение работы с базой.

- **commit()** — Сохраняет изменения в базе данных
- **close()** — Закрывает соединение с базой

🔗 Шаг 06:

Миссия 6

Формулировка: Сопоставьте SQL-команды с их назначением.

Понятия:

Ответ:

- SELECT
- INSERT
- UPDATE
- DELETE

Определения:

1. Извлекает данные из таблицы
2. Добавляет новую запись в таблицу
3. Изменяет существующую запись
4. Удаляет запись из таблицы

Подсказка: Вспомни базовые операции CRUD: Create, Read, Update, Delete.

- **SELECT** — Извлекает данные из таблицы
- **INSERT** — Добавляет новую запись в таблицу
- **UPDATE** — Изменяет существующую запись
- **DELETE** — Удаляет запись из таблицы

🔗 Шаг 07:

Миссия 7

Вопрос: Расположите шаги создания таблицы артефактов в SQLite:

1. Создать соединение с базой
2. Создать объект Cursor
3. Выполнить SQL-запрос на создание таблицы
4. Сохранить изменения через commit()
5. Закрыть соединение

🔗 **Подсказка:**

Сначала открываем базу, потом работаем с курсором, выполняем команду, сохраняем изменения и закрываем соединение.

Ответ:

- Создать соединение с базой
- Создать объект Cursor
- Выполнить SQL-запрос на создание таблицы
- Сохранить изменения через commit()
- Закрыть соединение

🔗 Шаг 08:

Миссия 8

Вопрос: Расположите шаги добавления новой записи в таблицу:

1. Создать соединение и курсор
2. Выполнить INSERT-запрос

3. Сохранить изменения `commit()`
4. Закрыть соединение

 Подсказка:

Сначала подготовка, потом запись, затем сохранение и закрытие.

Ответ:

- Создать соединение и курсор
- Выполнить `INSERT`-запрос
- Сохранить изменения `commit()`
- Закрыть соединение

 Шаг 09:

Миссия 9

Вопрос: Расположите шаги извлечения данных из таблицы:

1. Создать соединение и курсор
2. Выполнить `SELECT`-запрос
3. Получить результат через `fetchall()` или `fetchone()`
4. Закрыть соединение

 Подсказка:

Сначала подключение, затем выполнение запроса, получение данных и закрытие базы.

Ответ:

- Создать соединение и курсор
- Выполнить `SELECT`-запрос
- Получить результат через `fetchall()` или `fetchone()`
- Закрыть соединение

 Шаг 10:

Миссия 10

Вопрос: Расположите шаги обновления и удаления артефактов в SQLite:

1. Создать соединение и курсор
2. Выполнить `UPDATE` или `DELETE`-запрос
3. Сохранить изменения `commit()`
4. Закрыть соединение

 Подсказка:

Сначала подключаемся, затем меняем или удаляем данные, фиксируем изменения и закрываем базу.

Ответ:

- Создать соединение и курсор
- Выполнить `UPDATE` или `DELETE`-запрос
- Сохранить изменения `commit()`
- Закрыть соединение

Урок 3 - Pathlib

Шаг 01:

Миссия 1

Ты исследуешь древние каталоги артефактов. Какой модуль Python предназначен для работы с путями к файлам и папкам?

Подсказка: Этот модуль современный и объектно-ориентированный.

Ответ:

- **pathlib**
- os
- shutil
- glob

Шаг 02:

Миссия 2

Ты хочешь создать объект пути к файлу 'artefact.txt'. Какой вариант правильный?

Подсказка: Path нужно импортировать из pathlib.

Ответ:

- **Path('artefact.txt')**
- pathlib.file('artefact.txt')
- Pathlib('artefact.txt')
- os.path('artefact.txt')

Шаг 03:

Миссия 3

Какой метод Path используется для проверки существования файла?

Подсказка: Этот метод возвращает True, если файл или папка существует.

Ответ:

- **exists()**
- is_file()
- mkdir()
- resolve()

Шаг 04:

Миссия 4

Формулировка: Сопоставьте методы Path с их назначением.

Понятия:

Ответ:

- is_file()
- is_dir()
- mkdir()
- unlink()

Определения:

1. Проверяет, является ли путь файлом
2. Проверяет, является ли путь директорией
3. Создаёт новую директорию
4. Удаляет файл

Подсказка: Файлы и папки проверяются отдельно, создание и удаление — разные методы.

- **is_file()** — Проверяет, является ли путь файлом
- **is_dir()** — Проверяет, является ли путь директорией
- **mkdir()** — Создаёт новую директорию
- **unlink()** — Удаляет файл

Шаг 05:

Миссия 5

Формулировка: Сопоставьте свойства Path с их назначением.

Понятия:

Ответ:

- name
- stem
- suffix
- parent

Определения:

1. Имя файла с расширением
2. Имя файла без расширения
3. Расширение файла
4. Путь к родительской директории

Подсказка: Подумай, что возвращает каждый атрибут: имя, расширение, папка.

- **name** — Имя файла с расширением
- **stem** — Имя файла без расширения
- **suffix** — Расширение файла
- **parent** — Путь к родительской директории

Шаг 06:

Миссия 6

Формулировка: Сопоставьте методы для навигации по файлам и папкам с их действием.

Понятия:

Ответ:

- iterdir()
- glob('*.*txt')
- rglob('*.*log')

Определения:

1. Перечисляет все элементы в текущей директории
2. Находит файлы с указанным шаблоном в текущей директории

3. Находит файлы с указанным шаблоном рекурсивно в поддиректориях

Подсказка: Glob работает с шаблонами, rglob — рекурсивно.

- **iterdir()** — Перечисляет все элементы в текущей директории
- **glob('*.txt')** — Находит файлы с указанным шаблоном в текущей директории
- **rglob('*.log')** — Находит файлы с указанным шаблоном рекурсивно в поддиректориях

Шаг 07:

Миссия 7

Вопрос: Расположите шаги для создания нового файла и записи в него текста с помощью Path.

1. Создать объект Path для файла
2. Открыть файл через open() с режимом записи
3. Записать текст в файл
4. Закрыть файл

Подсказка:

Сначала путь, потом открытие, запись, закрытие — это стандартный порядок работы с файлами.

Ответ:

- Создать объект Path для файла
- Открыть файл через open() с режимом записи
- Записать текст в файл
- Закрыть файл

Шаг 08:

Миссия 8

Вопрос: Расположите шаги для чтения всех строк из текстового файла через Path.

1. Создать объект Path для файла
2. Открыть файл с режимом чтения
3. Вызвать метод readlines() или read_text()
4. Закрыть файл

Подсказка: Подумай о правильной последовательности: путь, открытие, чтение, закрытие.

Ответ:

- Создать объект Path для файла
- Открыть файл с режимом чтения
- Вызвать метод readlines() или read_text()
- Закрыть файл

Шаг 09:

Миссия 9

Вопрос: Расположите шаги для обхода всех файлов .txt в директории и поддиректориях:

1. Создать объект Path для директории
2. Вызвать метод rglob('*.txt')
3. Перебрать все найденные файлы в цикле
4. Обработать каждый файл

Подсказка:

Сначала путь, потом рекурсивный поиск, затем перебор и обработка файлов.

Ответ:

- Создать объект Path для директории
- Вызвать метод rglob('*.txt')
- Перебрать все найденные файлы в цикле
- Обработать каждый файл

Шаг 10:

Миссия 10

Вопрос: Расположите шаги для создания структуры папок и вложенных файлов:

1. Создать объект Path для корневой директории
2. Вызвать mkdir(parents=True, exist_ok=True)
3. Создать объекты Path для вложенных файлов
4. Создать файлы через open() и запись

Подсказка:

Сначала создаём директорию (включая родительские), потом пути для файлов, затем сами файлы.

Ответ:

- Создать объект Path для корневой директории
- Вызвать mkdir(parents=True, exist_ok=True)
- Создать объекты Path для вложенных файлов
- Создать файлы через open() и запись

Модуль 6 - Магия ООП — Создание персонажей

Урок 1 - Наследование

Шаг 01:

Миссия 1

Вы новичок в магическом мире Python. Вам нужно создать первого персонажа: обычного воина. Какой класс лучше всего описывает базового воина?

Подсказка: Начните с простого и универсального персонажа, чтобы в дальнейшем от него наследоваться.

Ответ:

- **class Warrior: pass**
- class Warrior(Wizard): pass
- def Warrior(): pass
- Warrior = 'fighter'

Шаг 02:

Миссия 2

Теперь магический мир расширяется. Вы хотите создать класс Маг, который тоже будет персонажем. Что нужно добавить, чтобы Маг был подобен Воину, но с магическими способностями?

Подсказка: Используйте наследование, чтобы не писать заново всё, что уже есть у Воина.

Ответ:

- **class Mage(Warrior): pass**
- class Mage: pass
- def Mage(Warrior): pass
- Mage = Warrior + 'magic'

🧙 Шаг 03:

Миссия 3

Вы хотите добавить уникальные способности Магу, например, заклинание огненного шара. Как правильно это сделать?

Подсказка: Метод в классе — это действие персонажа, которое можно вызвать.

Ответ:

- **class Mage(Warrior): def fireball(self): print('🔥 Огненный шар!')**
- class Mage(Warrior): fireball = '🔥 Огненный шар!'
- def fireball(): print('🔥 Огненный шар!')
- Mage.fireball = print('🔥 Огненный шар!')

🧙 Шаг 04:

Миссия 4

Формулировка: Сопоставьте типы персонажей с их особенностями.

Персонажи:

Ответ:

- Воин
- Маг
- Лучник
- Жрец

Особенности:

1. Близкий бой, высокая защита
2. Дальний бой с луком, ловкость
3. Заклинания, магическая атака
4. Лечение союзников, поддержка

🧙 Подсказка:

Подумайте, кто ближний боец, кто дальний, а кто помогает команде.

- **Воин** — Близкий бой, высокая защита
- **Лучник** — Дальний бой с луком, ловкость
- **Маг** — Заклинания, магическая атака
- **Жрец** — Лечение союзников, поддержка

🧙 Шаг 05:

Миссия 5

Формулировка: Сопоставьте методы персонажей с их действием.

Методы:

Ответ:

- `attack()`
- `heal()`
- `shoot_arrow()`
- `cast_spell()`

Действия:

1. Наносит урон в ближнем бою
2. Восстанавливает здоровье союзника
3. Стреляет стрелой на расстоянии
4. Использует магическое заклинание

 Подсказка:

Сопоставьте метод с логикой его действия: атакующий, поддерживающий или магический.

- **`attack()`** — Наносит урон в ближнем бою
- **`heal()`** — Восстанавливает здоровье союзника
- **`shoot_arrow()`** — Стреляет стрелой на расстоянии
- **`cast_spell()`** — Использует магическое заклинание

 Шаг 06:

Миссия 6

Формулировка: Сопоставьте ключевые слова Python с их применением при наследовании.

Ключевые слова:

Ответ:

- `super()`
- `class`
- `def`
- `pass`

Применение:

1. Объявление класса
2. Вызов метода родителя внутри дочернего класса
3. Определение функции или метода
4. Пустой блок кода

 Подсказка:

Подумайте, какое слово отвечает за объявление, какое за методы, а какое за обращение к родителю.

- **`class`** — Объявление класса
- **`super()`** — Вызов метода родителя внутри дочернего класса
- **`def`** — Определение функции или метода
- **`pass`** — Пустой блок кода

 Шаг 07:

Миссия 7: Расположите шаги создания нового персонажа с наследованием в правильном порядке:

1. Определение нового класса и указание родителя

2. Добавление уникальных методов и атрибутов
3. Создание объекта персонажа
4. Вызов методов объекта для проверки работы

 Подсказка:

Сначала создаем каркас класса, затем добавляем уникальное, потом объект и проверяем его работу.

Ответ:

- Определение нового класса и указание родителя
- Добавление уникальных методов и атрибутов
- Создание объекта персонажа
- Вызов методов объекта для проверки работы

 Шаг 08:

Миссия 8: Расположите шаги использования `super()` в дочернем классе:

1. Определение метода в дочернем классе
2. Вызов `super()` внутри метода
3. Передача аргументов методу родителя
4. Использование метода родителя внутри дочернего метода

 Подсказка:

Сначала нужно объявить метод, потом обратиться к родителю, передав нужные аргументы, и использовать результат.

Ответ:

- Определение метода в дочернем классе
- Вызов `super()` внутри метода
- Передача аргументов методу родителя
- Использование метода родителя внутри дочернего метода

 Шаг 09:

Миссия 9: Расположите этапы расширения персонажа с новыми способностями:

1. Анализ, какие способности нужны персонажу
2. Добавление новых методов и атрибутов в класс
3. Создание нового объекта класса
4. Тестирование новых возможностей объекта

 Подсказка:

Сначала понимаем, что добавить, потом реализуем в коде, создаем объект и проверяем работу.

Ответ:

- Анализ, какие способности нужны персонажу
- Добавление новых методов и атрибутов в класс
- Создание нового объекта класса
- Тестирование новых возможностей объекта

🔮 Шаг 10:

Миссия 10: Расположите шаги взаимодействия нескольких наследуемых персонажей в команде:

1. Создание объектов каждого персонажа
2. Вызов методов персонажей для совместного действия
3. Проверка совместимости действий (например, атака + лечение)
4. Анализ и корректировка методов при необходимости

🔮 Подсказка:

Сначала создаем объекты, потом проверяем их совместную работу и при необходимости улучшаем методы.

Ответ:

- Создание объектов каждого персонажа
- Вызов методов персонажей для совместного действия
- Проверка совместимости действий (например, атака + лечение)
- Анализ и корректировка методов при необходимости

📖 Урок 2 - Миксины

🔮 Шаг 01:

Миссия 1

Вы хотите создать персонажа с базовыми навыками. Как правильно подключить простой миксин для добавления нового поведения?

Подсказка: Миксин — это класс, который даёт дополнительные возможности без полной иерархии наследования.

Ответ:

- **class Character(AttackMixin): pass**
- class Character: AttackMixin = 'enabled'
- def Character(AttackMixin): pass
- Character = AttackMixin + 'skills'

🔮 Шаг 02:

Миссия 2

Вы хотите, чтобы ваш персонаж умел и атаковать, и лечить. Как лучше использовать миксины?

Подсказка: Миксины можно комбинировать, добавляя несколько возможностей одному персонажу.

Ответ:

- **class Hero(AttackMixin, HealMixin): pass**
- class Hero: AttackMixin, HealMixin = True, True
- def Hero(AttackMixin, HealMixin): pass
- Hero = AttackMixin + HealMixin

🔮 Шаг 03:

Миссия 3

Как правильно вызвать метод миксина в дочернем классе?

Подсказка: Метод миксина становится частью класса, его можно вызывать как обычный метод.

Ответ:

- **self.attack()**
- AttackMixin.attack(self)
- attack()
- super().attack()

Шаг 04:

Миссия 4

Формулировка: Сопоставьте миксин с его возможностью.

Миксины:

Ответ:

- AttackMixin
- HealMixin
- FlyMixin
- SwimMixin

Возможности:

1. Наносит урон врагу
2. Восстанавливает здоровье
3. Позволяет летать
4. Позволяет плавать

Подсказка:

Думайте по принципу: название миксина отражает его способность.

- **AttackMixin** — Наносит урон врагу
- **HealMixin** — Восстанавливает здоровье
- **FlyMixin** — Позволяет летать
- **SwimMixin** — Позволяет плавать

Шаг 05:

Миссия 5

Формулировка: Сопоставьте термины с их ролью при использовании миксинов.

Термины:

Ответ:

- Миксин
- Дочерний класс
- Метод миксина
- Комбинация миксинов

Роли:

1. Класс с дополнительными возможностями
2. Класс, который наследует миксин
3. Действие, которое добавляет миксин
4. Добавление нескольких возможностей одному персонажу

Подсказка:

Подумайте, кто даёт способности, а кто их получает.

- **Миксин** — Класс с дополнительными возможностями
- **Дочерний класс** — Класс, который наследует миксин
- **Метод миксина** — Действие, которое добавляет миксин
- **Комбинация миксинов** — Добавление нескольких возможностей одному персонажу

Шаг 06:

Миссия 6

Формулировка: Сопоставьте ошибки с их причиной при работе с миксинами.

Ошибки:

Ответ:

- Метод не найден
- Неправильный порядок наследования
- Дублирование методов
- Несовместимость миксинов

Причины:

1. Миксин не был добавлен в класс
2. Сложный порядок вызова `super()`
3. Два миксина имеют одинаковый метод
4. Некоторые миксины не могут работать вместе

Подсказка:

Подумайте, почему метод может быть недоступен и что происходит при множественном наследовании.

- **Метод не найден** — Миксин не был добавлен в класс
- **Неправильный порядок наследования** — Сложный порядок вызова `super()`
- **Дублирование методов** — Два миксина имеют одинаковый метод
- **Несовместимость миксинов** — Некоторые миксины не могут работать вместе

Шаг 07:

Миссия 7: Расположите шаги добавления нового миксина к персонажу:

1. Определение нового миксина с методами
2. Добавление миксина в список родителей класса
3. Создание объекта класса
4. Вызов методов миксина через объект

Подсказка:

Сначала создаём миксин, затем включаем его в класс, потом проверяем работу методов.

Ответ:

- Определение нового миксина с методами
- Добавление миксина в список родителей класса
- Создание объекта класса
- Вызов методов миксина через объект

Шаг 08:

Миссия 8: Расположите шаги комбинирования нескольких миксинов в одном классе:

1. Определение всех нужных миксинов
2. Указание их в списке родителей класса в нужном порядке
3. Создание объекта класса
4. Проверка работы методов каждого миксина

Подсказка:

Важен порядок наследования — методы вызываются сверху вниз.

Ответ:

- Определение всех нужных миксинов
- Указание их в списке родителей класса в нужном порядке
- Создание объекта класса
- Проверка работы методов каждого миксина

Шаг 09:

Миссия 9: Расположите шаги решения конфликта методов при использовании нескольких миксинов:

1. Определение, какой метод должен быть основным
2. Использование `super()` при вызове метода
3. Создание объекта класса
4. Проверка корректного выполнения методов

Подсказка:

Сначала решаем конфликт, потом создаём объект и тестируем.

Ответ:

- Определение, какой метод должен быть основным
- Использование `super()` при вызове метода
- Создание объекта класса
- Проверка корректного выполнения методов

Шаг 10:

Миссия 10: Расположите шаги создания полноценного персонажа с множеством миксинов:

1. Определение всех миксинов с уникальными методами
2. Создание класса персонажа с наследованием от миксинов
3. Создание объекта персонажа
4. Тестирование всех методов и возможностей объекта

Подсказка:

Сначала пишем миксины, затем класс, потом объект, затем проверяем работу всех функций.

Ответ:

- Определение всех миксинов с уникальными методами
- Создание класса персонажа с наследованием от миксинов
- Создание объекта персонажа
- Тестирование всех методов и возможностей объекта

Урок 3 - Перегрузка операторов

Шаг 01:

Миссия 1

Вы создаёте класс МагическийКуб. Как перегрузить оператор сложения + для объединения двух кубов?

Подсказка: В Python перегрузка операторов делается через специальные методы.

Ответ:

- `def __add__(self, other):`
- `def add(self, other):`
- `def + (self, other):`
- `def operator_plus(self, other):`

Шаг 02:

Миссия 2

Вы хотите, чтобы два персонажа при + объединялись в команду. Какой метод следует перегрузить?

Подсказка: Подумайте, какой специальный метод отвечает за +.

Ответ:

- `__add__`
- `__sub__`
- `__mul__`
- `__init__`

Шаг 03:

Миссия 3

Вы хотите выводить описание персонажа через `print()`. Какой метод нужно перегрузить?

Подсказка: `print()` автоматически вызывает специальный метод строкового представления объекта.

Ответ:

- `__str__`
- `__repr__`
- `__print__`
- `__display__`

Шаг 04:

Миссия 4

Формулировка: Сопоставьте оператор с его специальным методом в Python.

Операторы:

Ответ:

- +
- -
- *
- /

Методы:

1. `__add__`
2. `__sub__`
3. `__mul__`
4. `__truediv__`

 **Подсказка:**

Смотрите на символы и помните, как они называются в Python.

- `+` — `__add__`
- `-` — `__sub__`
- `*` — `__mul__`
- `/` — `__truediv__`

 **Шаг 05:**

Миссия 5

Формулировка: Сопоставьте перегруженные методы с их назначением для игровых объектов.

Методы:

Ответ:

- `__add__`
- `__sub__`
- `__eq__`
- `__str__`

Назначение:

1. Сравнение двух объектов на равенство
2. Объединение двух объектов
3. Вычитание одного объекта из другого
4. Вывод текстового описания объекта

 **Подсказка:**

Подумайте, что делает каждый оператор в обычной жизни и как это соотносить с методами.

- `__add__` — Объединение двух объектов
- `__sub__` — Вычитание одного объекта из другого
- `__eq__` — Сравнение двух объектов на равенство
- `__str__` — Вывод текстового описания объекта

 **Шаг 06:**

Миссия 6

Формулировка: Сопоставьте ситуацию с правильной перегрузкой оператора.

Ситуации:

Ответ:

- Сложение опыта двух персонажей
- Сравнение уровня персонажей
- Вычитание здоровья после атаки
- Вывод имени персонажа

Методы:

1. `__add__`
2. `__sub__`
3. `__eq__`
4. `__str__`

 Подсказка:

Сначала думайте, что соответствует арифметике и что нужно просто вывести или сравнить.

- **Сложение опыта двух персонажей** — `__add__`
- **Сравнение уровня персонажей** — `__eq__`
- **Вычитание здоровья после атаки** — `__sub__`
- **Вывод имени персонажа** — `__str__`

 Шаг 07:

Миссия 7: Расположите шаги перегрузки оператора `+` для игрового класса:

1. Создание метода `__add__` в классе
2. Определение логики сложения объектов
3. Создание двух объектов класса
4. Вызов оператора `+` между объектами

 Подсказка:

Сначала создаём метод, затем определяем логику, потом создаём объекты и пробуем сложение.

Ответ:

- Создание метода `__add__` в классе
- Определение логики сложения объектов
- Создание двух объектов класса
- Вызов оператора `+` между объектами

 Шаг 08:

Миссия 8: Расположите шаги перегрузки оператора `==` для игровых персонажей:

1. Создание метода `__eq__` в классе
2. Определение условий равенства
3. Создание двух объектов класса
4. Сравнение объектов через `==`

 Подсказка:

Сначала метод, потом логика, потом объекты, затем проверка через оператор.

Ответ:

- Создание метода `__eq__` в классе
- Определение условий равенства
- Создание двух объектов класса
- Сравнение объектов через `==`

 Шаг 09:

Миссия 9: Расположите шаги перегрузки оператора `-` для уменьшения здоровья после атаки:

1. Создание метода `__sub__` в классе

2. Определение логики уменьшения здоровья
3. Создание объектов атакующего и защищающегося
4. Применение оператора - между объектами

 Подсказка:

Подумайте, что сначала нужно описать метод, затем логику, потом объекты, затем тестирование.

Ответ:

- Создание метода `__sub__` в классе
- Определение логики уменьшения здоровья
- Создание объектов атакующего и защищающегося
- Применение оператора - между объектами

 Шаг 10:

Миссия 10: Расположите шаги создания полноценного класса персонажа с перегрузкой нескольких операторов:

1. Определение класса персонажа
2. Создание методов `__add__`, `__sub__`, `__eq__`, `__str__`
3. Создание объектов персонажей
4. Проверка работы всех операторов между объектами

 Подсказка:

Сначала класс, затем методы операторов, потом объекты, затем проверка функционала.

Ответ:

- Определение класса персонажа
- Создание методов `__add__`, `__sub__`, `__eq__`, `__str__`
- Создание объектов персонажей
- Проверка работы всех операторов между объектами

Модуль 7 - Финальный Босс — Оптимизация и Чистый Код

Урок 1 - Профилирование

 Шаг 01:

Миссия 1

Вы столкнулись с медленной функцией подсчёта опыта героев. Какой модуль в Python поможет измерить, сколько времени выполняется код?

Подсказка: Ищите инструмент для простого измерения времени выполнения.

Ответ:

- `time`
- `random`
- `os`
- `sys`

 Шаг 02:

Миссия 2

Вы хотите профилировать функцию с большим количеством вызовов. Какой модуль Python идеально подойдёт для подробного анализа всех функций?

Подсказка: Этот модуль создаёт статистику по количеству вызовов и времени работы функций.

Ответ:

- **cProfile**
- timeit
- logging
- random

Шаг 03:

Миссия 3

Вы хотите измерять скорость небольших операций в цикле для оптимизации боя с боссом. Какой инструмент лучше использовать?

Подсказка: Подумайте о мини-таймере, удобном для измерения небольших блоков кода.

Ответ:

- **timeit**
- cProfile
- trace
- pdb

Шаг 04:

Миссия 4

Формулировка: Сопоставьте модуль профилирования с его основной задачей.

Модули:

Ответ:

- time
- timeit
- cProfile

Назначение:

1. Подробный анализ вызовов функций и времени их выполнения
2. Измерение времени выполнения небольших фрагментов кода
3. Измерение общего времени работы программы или блока кода

Подсказка:

Смотрите на масштаб измерений: весь код, маленькие функции или сложная статистика?

- **time** — Измерение общего времени работы программы или блока кода
- **timeit** — Измерение времени выполнения небольших фрагментов кода
- **cProfile** — Подробный анализ вызовов функций и времени их выполнения

Шаг 05:

Миссия 5

Формулировка: Сопоставьте термин профилирования с его определением.

Термины:

Ответ:

- Профилирование
- Hotspot
- Benchmark
- Оптимизация

Определения:

1. Измерение производительности и времени работы программы
2. Участок кода, который выполняется дольше всего и требует внимания
3. Тест для измерения скорости работы кода
4. Изменение кода для ускорения работы и уменьшения использования ресурсов

🔗 Подсказка:

Подумайте, что важнее для анализа: измерять, находить проблемные места или улучшать код?

- **Профилирование** — Измерение производительности и времени работы программы
- **Hotspot** — Участок кода, который выполняется дольше всего и требует внимания
- **Benchmark** — Тест для измерения скорости работы кода
- **Оптимизация** — Изменение кода для ускорения работы и уменьшения использования ресурсов

🔗 Шаг 06:

Миссия 6

Формулировка: Сопоставьте шаг профилирования с его действием.

Шаги:

Ответ:

- Импорт модуля
- Запуск профилирования
- Анализ результатов
- Оптимизация кода

Действия:

1. Узнать, где код тратит больше всего времени
2. Добавить изменения для ускорения выполнения программы
3. Подключить нужный инструмент
4. Запустить код и собрать статистику

🔗 Подсказка:

Думайте о порядке действий: сначала подключаем инструмент, потом запускаем, затем анализируем и оптимизируем.

- **Импорт модуля** — Подключить нужный инструмент
- **Запуск профилирования** — Запустить код и собрать статистику
- **Анализ результатов** — Узнать, где код тратит больше всего времени
- **Оптимизация кода** — Добавить изменения для ускорения выполнения программы

🔗 Шаг 07:

Миссия 7: Расположите шаги анализа производительности функции с помощью timeit:

1. Импортировать модуль timeit

2. Определить код для тестирования
3. Запустить `timeit` для измерения времени выполнения
4. Проанализировать результаты и сделать выводы

🔗 **Подсказка:**

Сначала подключаем инструмент, потом описываем код, запускаем и анализируем.

Ответ:

- Импортировать модуль `timeit`
- Определить код для тестирования
- Запустить `timeit` для измерения времени выполнения
- Проанализировать результаты и сделать выводы

🔗 **Шаг 08:**

Миссия 8: Расположите шаги работы с `cProfile` для профилирования игры:

1. Импортировать `cProfile`
2. Создать функцию `main()` для запуска игрового цикла
3. Запустить профилирование `cProfile.run('main()')`
4. Проанализировать статистику вызовов функций

🔗 **Подсказка:**

Помните: сначала инструмент, потом функция, затем запуск и анализ.

Ответ:

- Импортировать `cProfile`
- Создать функцию `main()` для запуска игрового цикла
- Запустить профилирование `cProfile.run('main()')`
- Проанализировать статистику вызовов функций

🔗 **Шаг 09:**

Миссия 9: Расположите шаги оптимизации кода после профилирования:

1. **Найти hotspot** — медленные участки кода
2. Изменить алгоритм или структуру данных для ускорения
3. Протестировать изменённый код
4. Снова замерить время выполнения

🔗 **Подсказка:**

Сначала выявляем проблему, затем меняем код, тестируем и снова проверяем.

Ответ:

- **Найти hotspot** — медленные участки кода
- Изменить алгоритм или структуру данных для ускорения
- Протестировать изменённый код
- Снова замерить время выполнения

🔗 **Шаг 10:**

Миссия 10: Расположите полный цикл профилирования и оптимизации игрового кода:

1. Выбрать инструмент профилирования (`timeit` или `cProfile`)
2. Запустить код с профилированием

3. Проанализировать результаты и выявить медленные участки
4. Внести изменения в код для оптимизации
5. Повторно проверить производительность

💡 Подсказка:

Думайте о цикле: выбор инструмента → сбор данных → анализ → оптимизация → проверка.

Ответ:

- Выбрать инструмент профилирования (timeit или cProfile)
- Запустить код с профилированием
- Проанализировать результаты и выявить медленные участки
- Внести изменения в код для оптимизации
- Повторно проверить производительность

📖 Урок 2 - Dataclasses

💡 Шаг 01:

Миссия 1

Вы создаёте персонажа для игры и хотите, чтобы его свойства автоматически сохранялись. Какой декоратор позволит сделать это проще всего?

Подсказка: Он помогает быстро создавать классы с атрибутами и методами `__init__`.

Ответ:

- **@dataclass**
- @staticmethod
- @classmethod
- @property

💡 Шаг 02:

Миссия 2

Вы хотите, чтобы объекты класса персонажей могли автоматически сравниваться по значению всех полей. Какой параметр @dataclass позволяет это сделать?

Подсказка: Сравнение объектов важно для сортировки инвентаря и боя.

Ответ:

- **eq=True**
- order=True
- frozen=True
- init=False

💡 Шаг 03:

Миссия 3

Вы хотите создать неизменяемого персонажа, чтобы его характеристики нельзя было изменить после создания. Какой параметр dataclass использовать?

Подсказка: Идеально для константных объектов и сохранения баланса игры.

Ответ:

- **frozen=True**
- eq=True
- repr=False

- `order=True`

Шаг 04:

Миссия 4

Формулировка: Сопоставьте термин `dataclass` с его назначением.

Термины:

Ответ:

- `@dataclass`
- `frozen`
- `eq`
- `order`

Назначение:

1. Создаёт автоматически методы `__init__`, `__repr__` и другие
2. Запрещает изменение полей после создания объекта
3. Позволяет сравнивать объекты по значению полей
4. Позволяет сравнивать объекты по порядку (меньше/больше)

Подсказка:

Подумайте о том, что каждый параметр делает с объектами и их поведением.

- **`@dataclass`** — Создаёт автоматически методы `__init__`, `__repr__` и другие
- **`frozen`** — Запрещает изменение полей после создания объекта
- **`eq`** — Позволяет сравнивать объекты по значению полей
- **`order`** — Позволяет сравнивать объекты по порядку (меньше/больше)

Шаг 05:

Миссия 5

Формулировка: Сопоставьте методы `dataclass` с их эффектом.

Методы:

Ответ:

- `__init__`
- `__repr__`
- `__eq__`
- `__lt__`

Эффект:

1. Создаёт строковое представление объекта для удобного отображения
2. Сравнивает два объекта на равенство по значениям полей
3. Инициализирует объект с заданными полями
4. Позволяет сравнивать объекты по порядку (меньше/больше)

Подсказка:

Думайте о том, какие методы `dataclass` создаёт автоматически для вас.

- **`__init__`** — Инициализирует объект с заданными полями
- **`__repr__`** — Создаёт строковое представление объекта для удобного отображения
- **`__eq__`** — Сравнивает два объекта на равенство по значениям полей
- **`__lt__`** — Позволяет сравнивать объекты по порядку (меньше/больше)

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте пример использования dataclass с его назначением.

Примеры:

Ответ:

- Сохранение параметров персонажа
- Сравнение двух предметов инвентаря
- Создание неизменяемого объекта настроек
- Сортировка списка монстров по уровню

Назначение:

1. eq=True
2. frozen=True
3. order=True
4. Простое хранение данных с __init__

🔮 Подсказка:

Смотрите на контекст задачи и подумайте, какие возможности dataclass подходят.

- **Сохранение параметров персонажа** — Простое хранение данных с __init__
- **Сравнение двух предметов инвентаря** — eq=True
- **Создание неизменяемого объекта настроек** — frozen=True
- **Сортировка списка монстров по уровню** — order=True

🔮 Шаг 07:

Миссия 7

Расположите шаги создания и использования dataclass для персонажа в правильном порядке:

1. Импортировать dataclass из модуля dataclasses
2. Создать класс с @dataclass и описать поля
3. Создать объекты класса с нужными параметрами
4. Использовать объекты в игровом процессе

🔮 Подсказка:

Сначала импорт, затем определение класса, потом создание объектов и использование их в игре.

Ответ:

- Импортировать dataclass из модуля dataclasses
- Создать класс с @dataclass и описать поля
- Создать объекты класса с нужными параметрами
- Использовать объекты в игровом процессе

🔮 Шаг 08:

Миссия 8

Расположите шаги настройки сравнения и сортировки объектов dataclass:

1. Добавить eq=True для сравнения
2. Добавить order=True для возможности сортировки
3. Создать несколько объектов

4. Сравнить и отсортировать объекты

Подсказка:

Сначала включаем функции сравнения, создаём объекты, затем используем их.

Ответ:

- Добавить `eq=True` для сравнения
- Добавить `order=True` для возможности сортировки
- Создать несколько объектов
- Сравнить и отсортировать объекты

Шаг 09:

Миссия 9

Расположите шаги создания неизменяемого и сортируемого `dataclass` для настроек игры:

1. Создать класс с `@dataclass(frozen=True, order=True)`
2. Определить необходимые поля
3. Создать объекты и попробовать изменить поле (не должно работать)
4. Отсортировать список объектов по уровню сложности

Подсказка:

Сначала создаём класс с параметрами, потом поля, проверяем неизменяемость и сортируем.

Ответ:

- Создать класс с `@dataclass(frozen=True, order=True)`
- Определить необходимые поля
- Создать объекты и попробовать изменить поле (не должно работать)
- Отсортировать список объектов по уровню сложности

Шаг 10:

Миссия 10

Расположите полный цикл работы с `dataclass` для персонажей игры:

1. Импортировать `dataclass`
2. Создать класс с необходимыми параметрами (`eq, order, frozen`)
3. Создать объекты с нужными характеристиками
4. Сравнить и сортировать объекты
5. Использовать объекты в игровом процессе и сохранять данные

Подсказка:

Думайте о логике: импорт → создание класса → создание объектов → сравнение/сортировка → использование.

Ответ:

- Импортировать `dataclass`
- Создать класс с необходимыми параметрами (`eq, order, frozen`)
- Создать объекты с нужными характеристиками
- Сравнить и сортировать объекты
- Использовать объекты в игровом процессе и сохранять данные

Урок 3 - Чистый код

Шаг 01:

Миссия 1

Вы находите повторяющийся код в игре для подсчета очков. Какой принцип чистого кода поможет это исправить?

Подсказка: Подумайте, как сделать код DRY — не повторять одно и то же.

Ответ:

- **Вынести повторяющийся код в отдельную функцию**
- Добавить длинные комментарии
- Смешивать логику и интерфейс
- Использовать непонятные сокращения

Шаг 02:

Миссия 2

Вы заметили, что переменные имеют непонятные имена типа x1, temp2. Как лучше поступить?

Подсказка: Читаемый код легче поддерживать и развивать.

Ответ:

- **Дать переменным осмысленные имена**
- Оставить сокращения для компактности
- Убрать все имена переменных
- Использовать только цифры в именах

Шаг 03:

Миссия 3

Ваши функции становятся слишком длинными и сложными. Что нужно сделать?

Подсказка: Меньше — лучше, но не теряйте смысл.

Ответ:

- **Разделить функции на более мелкие логические части**
- Добавлять больше условий в одну функцию
- Писать функции без имен
- Убрать все комментарии

Шаг 04:

Миссия 4

Формулировка: Сопоставьте принцип чистого кода с его объяснением.

Принципы:

Ответ:

- DRY
- Короткие функции
- Осмысленные имена
- Комментарии

Объяснения:

1. Не повторяйся
2. Каждая функция делает одно действие и легко читается
3. Имена переменных и функций отражают их смысл
4. Объясняют, зачем сложная часть кода нужна

🔗 Подсказка:

Смотрите на назначение каждого принципа — читаемость, поддержка или уменьшение дублирования.

- **DRY** — Не повторяйся
- **Короткие функции** — Каждая функция делает одно действие и легко читается
- **Осмысленные имена** — Имена переменных и функций отражают их смысл
- **Комментарии** — Объясняют, зачем сложная часть кода нужна

🔗 Шаг 05:

Миссия 5

Формулировка: Сопоставьте проблему кода с её следствием.

Проблемы:

Ответ:

- Дублирование
- Сложные функции
- Плохие имена
- Отсутствие тестов

Следствия:

1. Трудно вносить изменения без ошибок
2. Повторение логики в разных местах
3. Непонятно, что делает код
4. Ошибки остаются незамеченными

🔗 Подсказка:

Подумайте, какое последствие логично вытекает из каждой проблемы.

- **Дублирование** — Повторение логики в разных местах
- **Сложные функции** — Трудно вносить изменения без ошибок
- **Плохие имена** — Непонятно, что делает код
- **Отсутствие тестов** — Ошибки остаются незамеченными

🔗 Шаг 06:

Миссия 6

Формулировка: Сопоставьте понятие с примером в Python.

Понятия:

Ответ:

- Функция одна задача
- DRY
- Понятные имена
- Комментарии

Примеры:

1. `def calc_score(): ...`
2. `def print_score(): ...; def print_score(): ...`
3. `player_name` вместо `x`
4. `# Эта функция считает очки`

Подсказка:

Смотрите на соответствие принципа и того, как он проявляется в коде.

- **Функция одна задача** — `def calc_score(): ...`
- **DRY** — `def print_score(): ...; def print_score(): ...`
- **Понятные имена** — `player_name` вместо `x`
- **Комментарии** — `# Эта функция считает очки`

Шаг 07:

Миссия 7: Расположите шаги исправления дублирующегося кода:

1. Найти повторяющийся участок
2. Вынести код в отдельную функцию
3. Заменить повторения вызовом функции
4. Протестировать, что всё работает

Подсказка:

Подумайте о логике: сначала обнаруживаем проблему, потом создаём решение, применяем и проверяем.

Ответ:

- Найти повторяющийся участок
- Вынести код в отдельную функцию
- Заменить повторения вызовом функции
- Протестировать, что всё работает

Шаг 08:

Миссия 8: Расположите шаги для улучшения читаемости функций:

1. Разбить функцию на маленькие части
2. Дать осмысленные имена переменным и функциям
3. Добавить комментарии, если нужно
4. Протестировать результат

Подсказка:

Логика: сначала делаем функции маленькими, потом делаем их понятными, добавляем комментарии и проверяем.

Ответ:

- Разбить функцию на маленькие части
- Дать осмысленные имена переменным и функциям
- Добавить комментарии, если нужно
- Протестировать результат

Шаг 09:

Миссия 9: Расположите этапы улучшения всего модуля игры:

1. Проанализировать код на дубли и непонятные части
2. Вынести повторяющиеся участки в функции
3. Дать осмысленные имена всем переменным и функциям
4. Добавить комментарии там, где нужно
5. Протестировать, что модуль работает корректно

Подсказка:

Сначала анализ, потом исправление, потом проверка.

Ответ:

- Проанализировать код на дубли и непонятные части
- Вынести повторяющиеся участки в функции
- Дать осмысленные имена всем переменным и функциям
- Добавить комментарии там, где нужно
- Протестировать, что модуль работает корректно

Шаг 10:

Миссия 10: Расположите полный цикл работы над чистотой кода в игре:

1. Анализ существующего кода
2. Исправление проблем с дублированием и сложными функциями
3. Дать понятные имена и добавить комментарии
4. Оптимизация структуры модулей и функций
5. Тестирование после изменений

Подсказка:

Думайте о цикле: анализ → исправление → улучшение → оптимизация → проверка.

Ответ:

- Анализ существующего кода
- Исправление проблем с дублированием и сложными функциями
- Дать понятные имена и добавить комментарии
- Оптимизация структуры модулей и функций
- Тестирование после изменений